Decidability of Non-Structural Subtype Entailment for a Single Constructor (Draft)

November 9, 2026

1 Introduction

A subtype constraint consists of inequalities $t \leq t'$ between terms built from type variables and type constructors. Satisfiability and entailment have been studied as basic problems for subtype constraints (see [1] and the references therein). The entailment problem $\varphi \models t \leq t'$ asks, for a constraint φ, whether $t \leq t'$ holds in all of its solutions, and it arises in the simplification of types in type inference systems with subtyping.

When types are represented by trees and the subtype order is an order that contains a least type $\bot$ and a greatest type $\top$, this order is called the non-structural subtype order (Section 3). In this order, trees of different shapes can also be compared. For example, $\bot \leq f(\bot, \top) \leq \top$. A constructor f is covariant if the order of each argument is preserved in the same direction (if $t_i \leq u_i$ holds for all i, then $f(\bar{t}) \leq f(\bar{u})$). In the order on trees in this paper the converse also holds: $f(\bar{t}) \leq f(\bar{u})$ is equivalent to having, for all i, $t_i \leq u_i$. The decidability of non-structural subtype entailment (NSSE) has been known as an open problem in the theory of programming languages [1]. The known lower bound was PSPACE-hardness, which holds even for the signature $\{\bot, f, \top\}$ (with the arity of f at least 2) [3]. Niehren and Priesnitz [1] reduced NSSE for the signature $\{\bot, f, \top\}$ to the universality problem for restricted cap set expressions, which are expressions denoting sets of words. Cap set expressions contain an operation that denotes the prefixes of repetitions of one word, and in general they denote languages that are not context-free. The decidability of their universality was also open. By contrast, the full first-order theory of the non-structural subtype order is undecidable [4].

Result of this paper. In this paper we show that, in the case of one constructor (of arity $d \geq 1$, the covariant f), NSSE is decidable for each of finite, regular and arbitrary trees (Theorem 6.27). The premise is a finite conjunction of inequalities and equalities between finite terms nested to arbitrary depth, and the query is an inequality between two finite terms. The core of the proof is a decision procedure for the universality of restricted cap set expressions, the target of the reduction of [1] (Theorem 6.25). The restricted cap set expressions of [1] impose, in addition to the form of the expression, a condition called shuffle. Our decision procedure does not use the shuffle condition, and it decides universality for all expressions of the same form (Section 4).

Outline of the method. The decision finally reduces to one inclusion test for regular languages and one evaluation of a first-order sentence over a finite alphabet. The following is a summary of the path to that point. Deciding whether a word is covered by a restricted cap set expression involves a comparison of two positions in the word (the border condition), and the set of covered words is not regular in general. In this paper we first set aside the words that end in the middle of an anchor or of the first root (Mid), and we split the remaining candidate words into those that can be extended infinitely within the language of candidates (live) and the others (nonlive). Whether all live words are covered turns out to be equivalent to coverage by roots of bounded length alone, and this reduces to one inclusion test for regular languages. For nonlive words, the maximum number of nontrivial borders of each suffix (the rank) is at most $\max(0, n_{\mathrm{acc}} - 2)$ (where n_{acc} is the number of accepting states of the DFA of the language of candidates). We then lower the rank one level at a time by block encoding, which splits a word into blocks using the last letter of the body as the separator, and along the way we maintain the property that all predicates that appear are represented by column-local automata. Finally, lowering (a parameter-uniform version of a result of Bès [2]), which translates a first-order sentence of an automatic structure into a formula over the alphabet one level below, reduces the problem to the evaluation of a sentence over a finite alphabet. Section 2 describes the flow of the proof in detail.

Formalization. The main theorem and the main parts of its proof are formalized in Lean 4 (Section 8). The formalization also independently re-proves the reduction of [1] (the forward direction), and it gives a procedure that computes a countermodel.

Scope. This paper treats the case of one covariant constructor. Decidability for a single constructor with arbitrary variance, and for several constructors with arbitrary variance, is outside the scope of this paper. Those results were obtained after the Japanese draft of this paper and are formalized in Lean in the same repository; they have not been externally reviewed.

Organization. Section 2 gives an overview of the proof. Section 3 gives preliminaries, and Section 4 states the reduction of [1]. Section 5 and Section 6 give the decision procedure for the universality of restricted cap set expressions, and Section 7 gives an example. Section 8 discusses the formalization, and Section 9 concludes.

2 Overview of the Proof

In this section, before going into details, we start from the goal of the proof and explain the main reductions in order. We use definitions informally here. Precise definitions and proofs are given from Section 3 on.

2.1 From entailment to universality

The subject is non-structural subtype entailment in the case of one constructor (of arity $d \geq 1$, the covariant f). The premise may contain inequalities and equalities of arbitrarily nested terms, and the query is an inequality of two terms. By flattening, these can be reduced to a constraint φ consisting of four kinds of literals on variables, and a query $x \leq y$ between variables. We treat three classes of trees: finite trees, regular trees and arbitrary trees. Paths of trees are represented by words over $\Sigma = \{0, \ldots, d-1\}$.

Niehren and Priesnitz [1] reduced this entailment to the following form (Section 4). First, decide the satisfiability of φ. If φ is not satisfiable, the entailment holds trivially. If it is satisfiable, then from φ and the query $x \leq y$ one can compute two restricted cap set expressions E_ℓ, E_r, and

$$\varphi \models x \leq y \iff E_\ell = \Sigma^* \text{ and } E_r = \Sigma^*$$

holds. The differences between the classes of trees appear only in the satisfiability test and in the construction of the countermodel. Deciding the universality of E_ℓ, E_r does not depend on the class of trees. Therefore the center of this paper is the following problem.

> Given an alphabet Σ and a restricted cap set expression E over it, decide whether $E = \Sigma^*$ (whether E is universal).

A word not belonging to E is called an avoider. When φ is satisfiable, an avoider corresponds to a path at which a countermodel of the original entailment violates $x \leq y$.

Destination. The decision finally reduces to two computations. One is an inclusion test for regular languages. The other is the evaluation of a first-order sentence over the finite alphabet Σ. The quantifiers of this sentence range not over words but over single letters of Σ, so it can be evaluated by checking finitely many cases. Below we describe how these two are reached.

2.2 The difficulty of deciding universality

A restricted cap set expression has the form

$$E = \bigcup_i \mathrm{pr}(A_i B_i^\circ) \cup \bigcup_j C_j \Sigma^*, \qquad B^\circ = \bigcup_{b \in B} \mathrm{pr}(b^*)$$

Here A_i, B_i, C_j are regular languages and $\mathrm{pr}(L)$ is the set of all prefixes of words of L. $\mathrm{pr}(A_i B_i^\circ)$ is the set of all prefixes of "words consisting of a word of A_i followed by repetitions of one $b \in B_i$". B° differs from the usual iteration B^* and keeps choosing the same b. For example, if $B = \{0, 1\}$, then $B^\circ = 0^* \cup 1^*$ and $01 \notin B^\circ$. A component with $B_i = \emptyset$ gives the empty set, so we remove such components in advance.

The set of all words belonging to none of the $C_j \Sigma^*$ is denoted Cand. The set of all prefixes up to the completion of an anchor (a word of A_i) and all prefixes up to the completion of the first root (a word of B_i), in both cases including the point of completion, is denoted Mid. Thus Mid consists of the words of $\mathrm{pr}(A_i)$ and of $A_i \,\mathrm{pr}(B_i)$, over all i. Words of Mid are always covered: since $B_i \neq \emptyset$, each such word is a prefix of some word of $A_i B_i^\circ$. As shown in Subsection 5.1,

$$E = \Sigma^* \iff \text{Cand} \setminus \text{Mid contains only words } w \text{ that have a cut}$$

holds. Here a cut is a pair of positions that, for some i and $0 \le j < t < |w|$, satisfies

$$w[:j] \in A_i \text{ (anchor)}, \qquad w[j{:}t] \in B_i \text{ (root)}, \qquad w[t:] \sqsubseteq w[j:] \text{ (border condition)}$$

namely the pair (j, t) ($\sqsubseteq$ is the prefix relation). Since $w[t:]$ and $w[j:]$ both end at the end of w, the border condition can be restated as "$w[t:]$ is a nontrivial border of $w[j:]$".

The anchor and root conditions are regular. The border condition, on the other hand, compares two distant places in the same word, and the language of covered words is not regular in general. The border condition is the only non-regular element of this problem. The whole of the proof below is devoted to giving a way to handle this condition by a finite computation.

2.3 Processing in three stages

We process the words of Cand in the following three stages.

(1) Mid These are covered without a cut, so there is no need to examine them.

(2) Live This is the set of all words that can be extended infinitely within Cand. Words of Mid have already been processed, so we consider the words of Live $\setminus$ Mid. The key is the observation that, assuming all live words are covered, the only way for such a word to extend while remaining covered is to repeat a root that is already visible (a ray). Hence the extension eventually follows a short cycle of the DFA of Cand. As a result, the root can be replaced by one of length at most $n_{\mathrm{acc}} n_{\mathrm{root}}$. Here n_{acc} is the number of accepting states of the DFA of Cand, and n_{root} is the maximum number of states of the DFAs of the B_i (but at least 1). The coverage ShortCov obtained by cutting root lengths at this bound (it contains Mid) is regular, and whether the live part is covered reduces to one inclusion test Live $\subseteq$ ShortCov (Subsection 5.2). If the inclusion fails, an avoider exists.

(3) Dead Let Dead = Cand $\setminus$ (Live $\cup$ Mid). If the inclusion in (2) holds, avoiders exist only in Dead. A word of Dead is a word every infinite extension of which eventually falls into some $C_j\Sigma^*$. Here periodicity is not forced. Instead, the nesting depth of the border condition is bounded. That is, for one left position j, the maximum number, in the range $j < t < |w|$, of right positions t satisfying the border condition (the rank) is at most $\max(0, n_{\mathrm{acc}} - 2)$ (Subsection 5.3). The key is that a short border of a word is also a border of a longer border, so that the positions of borders are nested. With too many borders, the DFA states at those positions repeat and the word can be extended infinitely. Dead is reduced, by a bounded number of rounds of block encoding and lowering, to the evaluation of a first-order sentence over a finite alphabet (see below).

2.4 Handling Dead: peeling off borders one level at a time

We treat the question of whether Dead contains an avoider as a problem of the following general form (the avoidance problem) (Section 6). A candidate word consists of its first letter (the seed) and the rest (the body). At the lowest level, for a word w of Dead, we prepend a dummy letter σ, take the result σw as the candidate word, and take all of w as the body. A row is a pair of a reader of words containing the markers L, R, namely an automaton T, and a guard G, which is a relation between two letters. T reads the candidate word with L inserted at the left cut position and R inserted at the right cut position. The dummy letter σ guarantees that even a cut at the very beginning of w has a letter immediately before it, to which the guard can refer. A row fires if, in addition to a condition of the same form as a cut of a cap (acceptance by T and the border condition), the guard holds for the letters immediately before the two cut positions. A cap term can be represented as a row whose guard is always true, and "words on which no row fires" are exactly the avoiders.

The seed and the guard are introduced because they are needed after the following block encoding. Block encoding splits the body into blocks at the occurrences of its last letter a, and makes a new word in which each block is one letter (the first block becomes the new seed). Then the border condition splits into "the two blocks containing the two cut positions end with a common suffix" and "the block sequences after them coincide". The former is a condition between two blocks (two letters of the new word), so it goes into the new guard. The latter remains as a border condition of the new word, but the nesting becomes one level shallower. If the new body is nonempty, the rank drops by at least 1, and if it is empty, the word is treated as a word consisting of a seed only (SeedOnly). The two letters compared by the new guard are the letters immediately before the new cut positions, that is, the two blocks containing the original cut positions.

In the original cap terms the right cut position was before the end of the word (the tail was nonempty), but in encoded words there arise firings where the right cut position is at the end, so the generalized problem allows these too. For a firing with empty tail (the right cut position at the end of the word) the border condition is trivial, and if moreover the last letter a of the body is fixed as a parameter (slice), the right argument of

the guard is determined and it becomes a condition on one letter. Therefore a firing with empty tail can be represented by an automaton, and it can be subtracted from the candidates at each level. After repeating block encoding at most $h = \max(0, n_{\mathrm{acc}} - 2)$ times, a firing with nonempty tail can no longer occur. Schematically, as long as the encoding continues,

$$\sigma w \text{ fires} \iff F^0(\sigma w) \vee F^1(\mathrm{enc}^1(\sigma w)) \vee \cdots \vee F^h(\mathrm{enc}^h(\sigma w))$$

holds (F^k is the automaton representing a firing with empty tail at level k, and enc^k is k rounds of block encoding). No border condition remains on the right-hand side. The detailed correspondence at each level is summarized in the table of Subsection 6.1.

2.5 Why it is decidable

Each time the level goes up, the alphabet becomes the words (blocks) of the level below, and it becomes an infinite set. Decidability is nevertheless obtained because the following invariant is maintained.

> The predicates that appear at level $k+1$ (the behavior of T on one block, the guard H, and so on) are all represented by synchronous automata that compare the letters of level k in a column-local way (only between letters in the same column).

Under this condition, a first-order sentence of level $k+1$ can be translated into a formula of level k while leaving the parameters as free variables (lowering, Subsection 6.4). This is a version of the result of Bès [2] made uniform in the parameters. Composing the translations in order from level h, one finally gets a sentence over the finite alphabet Σ, and this can be evaluated. Nothing is evaluated at intermediate levels. The whole is a procedure that "builds a formula and then evaluates it only once".

2.6 Example

Section 7 follows to the end the case $\Sigma = \{0, 1\}$, $A = 0^*$, $B = \{10\}$, $C = 0^*1\Sigma^3$. Dead is $0^m\{11, 100, 101, 110, 111\}$. When 0^m101 is encoded as the word $(0^m \mid 0)$, a firing with an empty tail occurs at level 1, and the word is removed from the candidates (it is covered). When 0^m11 is encoded as the word $(0^m \mid \varepsilon)$, it does not fire at level 1, and it becomes a word consisting of a seed only at level 2 (an avoider). Therefore this E is not universal, and the shortest avoider is 11.

2.7 Organization of this paper

Section 3	Preliminaries
Section 4	From entailment to universality of restricted cap set expressions (the result of [1])
Section 5	Analysis by regular languages (characterization by cuts, the live part, the rank of the nonlive part)
Section 6	Decision of the nonlive part (rows and firing, block encoding, lowering and the tower, proof of the main theorem)
Section 7	Example
Section 8	Formalization in Lean

3 Preliminaries

3.1 Trees and the non-structural subtype order

Fix an arity $d \geq 1$ and let the alphabet of paths be $\Sigma = \{0, \ldots, d-1\}$. A tree is a map $t : \Sigma^* \to \{\mathrm{B}, \mathrm{F}, \mathrm{T}\}$ satisfying the absorption law

$$t(p) \in \{\mathrm{B}, \mathrm{T}\} \implies t(pq) = t(p) \quad (q \in \Sigma^*)$$

B represents $\bot$, T represents $\top$, and F represents a node, with d children, of f. The label of a leaf is regarded as propagating to all of its (nonexistent) descendants. The constructor is defined by $f(t_0, \ldots, t_{d-1})(\varepsilon) = \mathrm{F}$, $f(t_0, \ldots, t_{d-1})(ip) = t_i(p)$. The projection to a path p is defined by $(\mathrm{Proj}_p t)(q) = t(pq)$.

The non-structural subtype order is the pointwise order given by $\mathrm{B} < \mathrm{F} < \mathrm{T}$:

$$t \leq u \iff \forall p \in \Sigma^*.\ t(p) \leq u(p)$$

A tree with finitely many F nodes is called a finite tree. A tree obtained by unfolding a graph consisting of a finite state set S, a labeling map $\mathrm{lab} : S \to \{\mathrm{B}, \mathrm{F}, \mathrm{T}\}$, a transition map $\delta : S \times \Sigma \to S$ and a root state is called a regular tree. Here, at a state labeled B or T, all descendants get the same label, in accordance with the absorption law. The classes of finite, regular and arbitrary trees are written Fin, Reg and All, respectively.

3.2 Constraints and entailment

Terms are built from variables, $\bot$, $\top$ and $f(\cdot,\dots,\cdot)$. A constraint φ is a finite conjunction of inequalities $a \le b$ and equalities $a = b$ between terms. When a valuation with values in a class of trees κ, call it ρ, satisfies all literals of φ, we write $\rho \models \varphi$. We define entailment by

$$\varphi \models_\kappa a \le b \iff \forall \rho \, \big(\rho \models \varphi \implies \rho(a) \le \rho(b)\big)$$

If φ is not satisfiable, the entailment holds trivially.

By flattening, a constraint can be reduced to one consisting of the four kinds of literals on variables

$$x \le f(\bar{y}), \qquad f(\bar{y}) \le x, \qquad x = \bot, \qquad x = \top$$

(the four-form), and a query can be reduced to $x \le y$ between variables ([1], Section 4). Concretely, to each subterm t we assign a new variable z_t and add $z_{f(\bar{t})} \le f(\bar{z}_{t_i})$ and $f(\bar{z}_{t_i}) \le z_{f(\bar{t})}$ (by antisymmetry, z_t takes the value of t). An inequality $u \le v$ between variables is replaced, using new variables h, e, by $f(u,e,\dots,e) \le h \le f(v,e,\dots,e)$ (by decomposition $u \le v$ follows, and conversely, if $u \le v$, it is satisfied by $e = \bot$, $h = f(u,\bot,\dots,\bot)$). An equality is split into two inequalities. For example, take $d = 2$ and consider the constraint $x \le f(y,\bot)$ and the query $x \le y$. If to the subterm $\bot$ we assign a variable $z_\bot$ and set $z_\bot = \bot$, the constraint becomes the four-form $x \le f(y, z_\bot)$, $z_\bot = \bot$, and the query stays $x \le y$. This reduction preserves each class $\kappa \in \{\mathrm{Fin}, \mathrm{Reg}, \mathrm{All}\}$. That is, an original solution extends to a solution of the transformed constraint, and a solution of the transformed constraint restricts on the original variables to an original solution, so the truth of the entailment does not change.

3.3 Words

Over Σ, the length of a word w is written $|w|$, and positions are counted from 0. $w[j{:}t]$ is the factor from position j up to just before t, and we set $w[{:}j] = w[0{:}j]$, $w[j{:}] = w[j{:}|w|]$. The fact that a word u is a prefix of a word v is written $u \sqsubseteq v$. The infinite repetition of a nonempty word r is written r^ω, and $\sqsubseteq$ is also used for finite prefixes of infinite words. For a language or infinite word L, the set of all its finite prefixes is written $\mathrm{pr}(L)$.

A border of a word z is a word that is both a prefix and a suffix of z. The empty word and z itself are always borders, and the other borders are called nontrivial borders. A word x has period $p \ge 1$ if, for $0 \le k < |x| - p$, we have $x[k] = x[k+p]$. Having a nontrivial border of length ℓ ($0 < \ell < |x|$) is equivalent to having period $|x| - \ell$.

3.4 Structures and symbolic automatons

From Subsection 6.2 on, alphabets become infinite sets. We therefore use the following generalization of finite automata.

Structures and formulas. A structure M is a structure for a signature consisting of finitely many relation symbols and constant symbols, and its domain is written D. Equality is always available as a logical symbol. A first-order formula of M is simply called a formula of M. A formula may have as free variables variables standing for letters $(x, y, \dots)$ and parameters $\bar{p}$. A structure whose domain is a finite set Σ and whose signature consists only of constant symbols for the letters is written $M = \Sigma$.

Definition 3.1 (symbolic automaton)**.** Let M be a structure and $\bar{p}$ a tuple of parameters. A symbolic automaton over M is a nondeterministic automaton with finitely many states and finitely many edges satisfying the following.

- Each edge e that reads a data letter carries as its label, with one letter variable x and the parameters $\bar{p}$ as free variables, an M formula $\theta_e(x;\bar{p})$. If $M \models \theta_e(c;\bar{p})$, this edge can read the letter $c \in D$. The letter variable x takes a separate value each time an edge is read, while the parameters $\bar{p}$ take a common value along a whole accepting path.
- With only the parameters $\bar{p}$ as free variables, formulas of M give the conditions for being an initial state and an accepting state.
- The automaton may have edges that read a fixed finite number of markers (special letters not in D). The conditions of those edges are also formulas in the parameters only.

The language accepted by a symbolic automaton without markers is called an M-regular language.

When $M = \Sigma$, each label denotes a subset of Σ, so a symbolic automaton is the same as an ordinary NFA. Even when D is infinite, a symbolic automaton is an object with a finite syntax. Below we use only the following two properties, and neither requires a procedure deciding the truth of formulas of M.

Lemma 3.2 (Syntactic Boolean closure)**.** *The languages accepted by symbolic automatons over M are closed under intersection, union and complement, and symbolic automatons accepting each of these can be built from the syntax alone.*

Proof. Union is by disjoint union and intersection is by the product construction, with labels combined by $\wedge$. Complement is by determinizing with the subset construction and then negating the acceptance condition. The new states are the subsets S of states (including the empty set). The condition for S to be initial is the formula expressing "the set of original states whose initial condition is true is exactly S", namely $\bigwedge_{q\in S} \mathrm{Init}_q(\bar{p}) \wedge \bigwedge_{q\notin S} \neg\mathrm{Init}_q(\bar{p})$. The condition for S to be accepting is $\bigvee_{q\in S} \mathrm{Fin}_q(\bar{p})$, and for the complement this is negated. For the data edges leaving S, consider, for the edges leaving elements of S, their labels $\theta_1, \ldots, \theta_n$ and the 2^n combinations $\bigwedge_k \pm\theta_k(x;\bar{p})$ that make each label true or false; we call these cells (Boolean cells). For each cell we add one edge to the set of targets of the edges whose labels are made true. The cells are mutually exclusive and together cover D, so the resulting automaton is deterministic and complete. For marker edges we do the same construction, for each marker, with cells of conditions in the parameters only. Unsatisfiable cells are kept without being decided. Such an edge simply cannot be used by any letter and does not affect the accepted language. □

Lemma 3.3 (Expressing nonemptiness by a formula)**.** *Let $\mathcal{A}$ be a symbolic automaton over M. With the parameters $\bar{p}$ as free variables, an M formula $\mathrm{NonEmpty}_{\mathcal{A}}(\bar{p})$ can be built from the syntax alone such that, for all $\bar{p}$, the language accepted by $\mathcal{A}$ is nonempty if and only if $M \models \mathrm{NonEmpty}_{\mathcal{A}}(\bar{p})$.*

Proof. There are finitely many simple paths (paths that do not pass through the same state twice; paths of length 0 are included) π from an initial state to an accepting state. Set

$$\mathrm{NonEmpty}_{\mathcal{A}}(\bar{p}) = \bigvee_{\pi} \Big(\mathrm{Init}_{\pi}(\bar{p}) \wedge \mathrm{Fin}_{\pi}(\bar{p}) \wedge \bigwedge_{e\in\pi} \exists x.\ \theta_e(x;\bar{p}) \Big)$$

Here Init_{π}, Fin_{π} are the conditions of the start and end of π, and for marker edges their condition is put in place of $\exists x.\ \theta_e$. For a path of length 0 the conjunction over edges is empty and is taken to be true. If some word is accepted, removing loops from its accepting path gives a simple path, and the label of each remaining edge is satisfied by the corresponding letter of the original word. Conversely, if the right-hand side is true, the word obtained by choosing, independently for each edge, a letter satisfying the edge's label, and concatenating them, is accepted along π. The choices can be made independently for each edge because each label constrains only the one letter read on that edge and the parameters. The parameters $\bar{p}$ have a common value along the whole path and are never quantified per edge. □

3.5 Synchronous multitape automata and automatic structures

To read, over D, a tuple of words $(u_1, \ldots, u_n)$ with one automaton, we use convolution. We reverse each u_k and, to make the lengths equal, append padding # (# $\notin D$) after the shorter ones. We then form the word whose k-th letter is $(\mathrm{rev}(u_1)[k], \ldots, \mathrm{rev}(u_n)[k])$. That is, the argument words are aligned at their right ends and read column by column from the right. The alignment at the right end serves, in Subsection 6.3, to place common suffixes in the same columns. Note that the symbolic automata of candidates and rows read words from the left. Only the automata representing relations with words as arguments (Run and H of Subsection 6.3) read right-aligned from the right.

Each column of a convolution is an element of $(D \cup \{\#\})^n$, and a column all of whose components are # does not occur. Also, on each tape no data letter occurs after #.

Definition 3.4 (column-local automata)**.** Over M, an n-tape synchronous automaton is an automaton reading the above convolution in which each edge carries a label of the following form. A label is a pair consisting of the set of indices of the tapes that hold the padding # in that column, $P \subsetneq \{1, \ldots, n\}$, and, with free variables only (for $k \notin P$) the variables x_k and the parameters $\bar{p}$, an M formula $\theta(\bar{x};\bar{p})$. This edge can read a column in which the set of indices of components equal to # is exactly P and whose remaining components $\bar{c}$ satisfy $M \models \theta(\bar{c};\bar{p})$. Then each edge relates only letters of the same column and the parameters. This property is called column-local.

Since labels are restricted to this form, a synchronous automaton is always column-local. M is infinite in general, and letters of distant columns cannot be remembered in the states. Therefore, for a predicate to be represented by a synchronous automaton, it must compare words only between letters of the same column (and finitely much state information). This is the point we check when representing each predicate by a synchronous automaton in Section 6 (Lemma 6.15), and the proof of lowering (Lemma 6.21) uses this property.

Definition 3.5 (automatic structure)**.** On the domain D^*, when each relation of a structure N is given by a synchronous automaton with parameters $\bar{p}$ over M, we call N an automatic structure (with parameters $\bar{p}$) over M and write $N[\bar{p}]$.

One element of N is a word of D, and it is passed to an automaton over M as a whole input word. For example, if $D = \{0, 1\}$ and $U = 010$, then U is one element in N, but for an automaton giving a relation on U over M it is an input of 3 letters. When $M = \Sigma$ is finite, this is a classical automatic structure (in the version that reads words reversed). The translation of first-order sentences of an automatic structure over M into formulas of M is stated in Subsection 6.4 (lowering).

4 From entailment to universality of restricted cap set expressions

In this section we state the reduction of Niehren and Priesnitz [1] from entailment to the universality problem, in the form used in this paper. The reduction theorem below is due to [1]. We refer to that paper for the details of the proof and only add the main points of the normalization used in this paper. This paper uses only the direction that reduces entailment to universality.

4.1 Cap automata and restricted cap set expressions

Definition 4.1 (cap automaton [1], Definition 1)**.** [1] allows a set of initial states and ε transitions, but here we state the case needed in this paper, with one initial state and no ε transitions. Given an alphabet Σ, a cap automaton P over it is a pair of a finite automaton over Σ (a state set Q, an initial state q_0, a set of accepting states and a transition relation) and a set of pairs of states called P-edges, $\mathcal{E} \subseteq Q \times Q$. The arrow $q \xrightarrow{u} q'$ means reading u by ordinary transitions only, going from q to q' (P-edges are not included). The language of P is

$$L(P) = \{\pi \mid q_0 \xrightarrow{\pi} q_f \ (q_f \text{ is an accepting state})\} \ \cup \ \{\pi\,\nu \mid \nu \in \mathrm{pr}(\mu^*),\ q_0 \xrightarrow{\pi} q_1 \xrightarrow{\mu} q_2,\ (q_2, q_1) \in \mathcal{E}\}$$

The second term expresses that, on a loop closed by the P-edge (q_2, q_1), a word μ may be repeated any number of times and stopped midway. [1] called expressions denoting languages of this form cap set expressions.

In this paper, we call an expression of the following form a restricted cap set expression (Subsection 5.1).

$$E = \bigcup_{i \in I} \mathrm{pr}(A_i B_i^\circ) \ \cup \ \bigcup_{j \in J} C_j \Sigma^*, \qquad B^\circ = \bigcup_{b \in B} \mathrm{pr}(b^*).$$

Here I, J are finite sets and A_i, B_i, C_j are regular languages.

Remark 4.2. A "restricted cap set expression" in Definition 4 of [1] is the same syntax as above with an added condition called shuffle (for each cap component satisfying $B_i \neq \{\varepsilon\}$, if $\pi \in A_i \cap A_i B_i$ then $\pi\Sigma^* \subseteq A_i$). Section 5 and Section 6 of this paper decide universality for all expressions of the above form, without assuming the shuffle condition. The shuffle condition is required in [1] for the reverse reduction (from universality to entailment), and it is not used in this paper.

4.2 The reduction

Consider a constraint φ in four-form and a query $x \leq y$ (Section 3).

Theorem 4.3 ([1], Theorem 2, Theorem 3, Proposition 14, 15)**.** *For each class $\kappa \in \{\mathrm{Fin}, \mathrm{Reg}, \mathrm{All}\}$, the following hold.*

1. *Whether φ is satisfiable in κ can be decided in polynomial time.*
2. *From φ and $x \leq y$, two cap automata P_ℓ, P_r (the left side and the right side) can be computed in polynomial time. If φ is satisfiable in κ, then*

   $$\varphi \models_\kappa x \leq y \iff L(P_\ell) = \Sigma^* \text{ and } L(P_r) = \Sigma^*$$

   holds.
3. *$L(P_\ell)$ and $L(P_r)$ can be represented as restricted cap set expressions of the above form, and the DFAs of their components A_i, B_i, C_j can be computed.*

(1) is by Theorem 3 of [1], (2) is by Theorem 2 and its proof, and (3) is by Proposition 14 (the constructed cap automaton is restricted) and the second half of the proof of Proposition 15 (the language of a restricted cap automaton is represented by an expression). In this reduction procedure, the differences between the classes of trees appear only in the satisfiability test of (1).

The meaning of the two problems, left and right. For trees t, u and a path p, there are only two ways in which $t(p) \leq u(p)$ can fail (since the labels are totally ordered by B < F < T).

- Left failure: $t(p) = \mathrm{T}$ and $u(p) \neq \mathrm{T}$.
- Right failure: $u(p) = \mathrm{B}$ and $t(p) \neq \mathrm{B}$.

If φ has no solution ρ with a left failure at a path p for $\rho(x), \rho(y)$, we say that p is left query-safe (and similarly for right). Since the order is a path-by-path comparison, $\varphi \models x \leq y$ is equivalent to "all paths are both left and right query-safe". When φ is satisfiable, P_ℓ (P_r) accepts exactly the left (right) query-safe paths. Therefore an avoider of E_r is a "path such that $\rho(y)(p) = \mathrm{B}$ and $\rho(x)(p) \neq \mathrm{B}$ for some solution ρ", where p is the path, and this is the position where the countermodel fails (and similarly for left). The words treated in the following sections are paths of trees in this sense.

Example 4.4 (Example 3 of [1])**.** Let $d = 2$, $\varphi = \{x \leq f(y,y),\ f(x,x) \leq y\}$, with the query $x \leq y$. This entailment does not hold. For example, $\rho(y) = f(f(\top, \bot), \top)$, $\rho(x) = f(f(\bot, \top), \bot)$ satisfy φ, but at the path 01 we have $\rho(x)(01) = \mathrm{T}$, $\rho(y)(01) = \mathrm{B}$, so $x \leq y$ fails. This failure is both a left failure and a right failure, so the path 01 is an avoider of both E_ℓ and E_r. On the other hand, every path in $0^* \cup 1^*$ is both left and right query-safe, and is covered by both expressions.

Main points of (3). The cap automaton P constructed by [1] has one initial state and no ε transitions, and moreover it has the following two properties ([1], Definition 3, Proposition 14).

- gap universal: if a non-accepting state moves to an accepting state by one letter, then from that accepting state all words are accepted.
- strictly cap: a state from which the source of a P-edge can be reached by a path of positive length is an accepting state.

The language of the finite automaton underlying P is denoted A, and we set

$$\mathrm{Core} = \{w \mid w \text{ has all its prefixes in } A\}, \qquad \mathrm{Ideal} = \{w \mid w\Sigma^* \subseteq A\}$$

From gap universal it follows that $A = \mathrm{Core} \cup \mathrm{Ideal}$, and from strictly cap it follows that the prefix closure of the contribution of each P-edge is contained in $L(P)$. As a result,

$$L(P) = \mathrm{pr}\big(\mathrm{Core}\,\{\varepsilon\}^\circ\big) \ \cup \bigcup_q \mathrm{pr}\big(A_q B_q^\circ\big) \ \cup \ \mathrm{Ideal}\,\Sigma^*$$

holds. Here q ranges over the targets of P-edges, and

$$A_q = \{u \mid q_0 \xrightarrow{u} q\}, \qquad B_q = \{v \mid \exists s.\ q \xrightarrow{v} s,\ (s,q) \in \mathcal{E}\}$$

Core is a cap component whose root language is $\{\varepsilon\}$, and $\mathrm{Ideal}\,\Sigma^*$ is a component of the form $C_j\Sigma^*$. In summary, ordinary acceptance by the underlying finite automaton splits into Core and Ideal. The contribution of acceptance via a P-edge is $A_q B_q^\circ$, and by strictly cap, after adding the prefixes accepted in the ordinary way (up to the middle of the anchor), it is collected, for each target q of a P-edge, into a cap component $\mathrm{pr}(A_q B_q^\circ)$.

4.3 Assembling the main theorem

By Theorem 4.3, to decide $\varphi \models_\kappa x \leq y$ it suffices to do the following.

1. If φ is not satisfiable in κ, the entailment holds.
2. If it is satisfiable, compute P_ℓ, P_r and represent their languages as restricted cap set expressions E_ℓ, E_r.
3. The entailment holds if and only if E_ℓ and E_r are both universal.

The decision in step 3 is the subject of Section 5 and Section 6 (Theorem 6.25).

Remark 4.5. [1] also claims the reverse reduction (reducing the universality of restricted restricted cap set expressions to entailment), but the result of this paper does not use the reverse reduction. Also, the reduction of this section (the forward direction) was independently re-proved in the course of writing this paper and is formalized in Lean (Section 8). The computation of countermodels is also given in that formalization.

5 Analysis by regular languages: cuts, the live part, and rank

In this section we reduce the universality of a restricted cap set expression E to one regular language inclusion test and one condition on the words of Dead. Let the cap part of E be $\mathrm{Cov} = \bigcup_i \mathrm{pr}(A_i B_i^\circ)$, and let the set of words that have no prefix in any of the C_j be Cand. Then the reduction is the following chain of equivalences.

$$\begin{aligned} E = \Sigma^* &\iff \mathrm{Cand} \subseteq \mathrm{Cov} \\ &\iff \mathrm{Live} \subseteq \mathrm{Cov} \ \wedge \ \mathrm{Dead} \subseteq \mathrm{Cov} \\ &\iff \mathrm{Live} \subseteq \mathrm{ShortCov} \ \wedge \ \big(\text{Dead consists of words with a cut}\big) \end{aligned}$$

The first equivalence follows from Lemma 5.3. The second follows from $\mathrm{Cand} = (\mathrm{Cand} \cap \mathrm{Mid}) \cup \mathrm{Live} \cup \mathrm{Dead}$ and $\mathrm{Mid} \subseteq \mathrm{Cov}$ (Lemma 5.3). The third follows, for Live, from Lemma 5.14, and for Dead, from Lemma 5.5 (words of Dead do not belong to Mid) (Corollary 5.15). Mid, Live, ShortCov, $\mathrm{Dead} = \mathrm{Cand} \setminus (\mathrm{Live} \cup \mathrm{Mid})$ are languages defined below. Together with Cand, they are all regular languages that can be constructed from the given DFAs. On the other hand, Cov is in general not regular (Remark 5.8), so the first inclusion $\mathrm{Cand} \subseteq \mathrm{Cov}$ cannot be treated directly as a regular language inclusion test. In the last line, on the side of Live, Cov is replaced by the regular ShortCov, and on the side of Dead, membership in Cov is restated as the existence of a cut. The former is a single regular language inclusion test. The latter test is carried out in Section 6. As a preparation for it, in this section we show that the rank of the words of Dead is bounded by the number of accepting states of the DFA.

In Section 5.1 we characterize the coverage of words outside Mid by the existence of a cut. In Section 5.2 we reduce the coverage test for live words to $\mathrm{Live} \subseteq \mathrm{ShortCov}$ and obtain the reduction above. In Section 5.3 we prove an upper bound on the rank of the words of Dead (the rank lemma).

5.1 Coverage test outside Mid

5.1.1 Setting

Let Σ be a finite alphabet. The notation for words follows Section 3. For a single word b we abbreviate $\mathrm{pr}(b) = \mathrm{pr}(\{b\})$.

Definition 5.1 (restricted cap set expression)**.** Let I and J be finite sets. Suppose that regular languages A_i, B_i $(i \in I)$ and C_j $(j \in J)$ are given by DFAs. Put $B^\circ = \bigcup_{b \in B} \mathrm{pr}(b^*)$, and call

$$E \ = \ \bigcup_{i \in I} \mathrm{pr}(A_i B_i^\circ) \ \cup \ \bigcup_{j \in J} C_j \Sigma^*$$

a restricted cap set expression. A component with $B_i = \emptyset$ gives the empty set, so we remove such components in advance. If $E = \Sigma^*$, we say that E is universal. A word that does not belong to E is called an avoider.

A word w belongs to $\mathrm{pr}(A_i B_i^\circ)$ if and only if for some $a \in A_i$, $b \in B_i$, $k \geq 0$ we have $w \sqsubseteq a b^k$.

Definition 5.2 (Cand, Cov, Mid)**.** We define the following three languages.

$$\mathrm{Cand} = \Sigma^* \setminus \bigcup_j C_j \Sigma^*, \qquad \mathrm{Cov} = \bigcup_i \mathrm{pr}(A_i B_i^\circ), \qquad \mathrm{Mid} = \bigcup_i \big(\mathrm{pr}(A_i) \cup A_i \,\mathrm{pr}(B_i)\big).$$

Lemma 5.3. *1.* $E = \Sigma^*$ *if and only if* $\mathrm{Cand} \subseteq \mathrm{Cov}$*.*

2. We have $\mathrm{Mid} \subseteq \mathrm{Cov}$*.*

3. Cand *and* Mid *are prefix-closed regular languages.*

Proof. (1) Σ^* is the disjoint union of Cand and $\bigcup_j C_j \Sigma^*$. The latter is contained in E by definition. Hence $E = \Sigma^*$ if and only if every word of Cand belongs to Cov.

(2) For every b we have $\varepsilon \in \mathrm{pr}(b^*)$, so $\varepsilon \in B_i^\circ$. Hence $A_i \subseteq A_i B_i^\circ$, and $\mathrm{pr}(A_i) \subseteq \mathrm{pr}(A_i B_i^\circ)$ holds. Also, for $b \in B_i$ we have $\mathrm{pr}(b) \subseteq \mathrm{pr}(b^*) \subseteq B_i^\circ$, so $A_i \,\mathrm{pr}(B_i) \subseteq A_i B_i^\circ \subseteq \mathrm{pr}(A_i B_i^\circ)$ holds.

(3) Let $u \notin \mathrm{Cand}$. Then for some j and $c \in C_j$ we have $u \in c\,\Sigma^*$. Every extension of u, say uv, also belongs to $c\,\Sigma^*$, so $uv \notin \mathrm{Cand}$. By contraposition, prefixes of words of Cand belong to Cand. The language $\mathrm{pr}(A_i)$ is prefix-closed by definition. If $a \in A_i$, $u \sqsubseteq b \in B_i$, then a prefix of au is either a prefix of a, or it can be written, using a prefix of u, say u', as au'. The former belongs to $\mathrm{pr}(A_i)$, and the latter to $A_i \,\mathrm{pr}(B_i)$. Hence Mid is prefix-closed. Regularity follows from the finiteness of I, J and the closure of regular languages under finite union, complement, concatenation and prefix closure. □

By Lemma 5.3 (1), the problem reduces to whether Cand contains a word that does not belong to Cov. By (2), such a word does not belong to Mid.

5.1.2 Characterization by cuts

Definition 5.4 (Cut)**.** For a word w and $i \in I$, a pair of positions (j, t), $0 \le j < t < |w|$, satisfying the following three conditions is called, with respect to component i, a cut of w.

(a) $w[:j] \in A_i$. This part is called the anchor.

(b) $w[j:t] \in B_i$. This part is called the root, and its length $t - j$ is called the period.

(c) $w[t:] \sqsubseteq w[j:]$. This condition is called the border condition, and $w[t:]$ is called the tail.

If for some $i \in I$ there exists, with respect to component i, a cut of w, we say that w has a cut.

Both $w[t:]$ and $w[j:]$ end at the end of w, so $w[t:]$ is always a suffix of $w[j:]$. Hence the border condition (c) means that $w[t:]$ is a border of $w[j:]$. Moreover, $j < t$ implies that $w[t:]$ is shorter than $w[j:]$, and $t < |w|$ implies that $w[t:]$ is nonempty. That is, (c) is equivalent to $w[t:]$ being a nontrivial border of $w[j:]$. This is also equivalent to $w[j:]$ having period $t - j$.

Lemma 5.5. *Let $w \notin$ Mid. Then $w \in$ Cov if and only if for some i there exists, with respect to component i, a cut of w.*

Proof. ($\Leftarrow$) Let (j, t) be a cut with respect to component i, and put $r = w[j:t]$. We have $w[j:] = r\, w[t:]$, and by (c), $w[t:] \sqsubseteq w[j:]$. In general, for a word x and a nonempty word r, if $x = ry$ and $y \sqsubseteq x$, then x is a prefix of r^ω. This can be shown by induction on $|x|$. Hence $w[j:]$ is a prefix of r^ω, and for sufficiently large k we have $w \sqsubseteq w[:j]\, r^k$. By (a) and (b), $w[:j] \in A_i$ and $r \in B_i$, so $w \in \mathrm{pr}(A_i B_i^\circ) \subseteq$ Cov.

($\Rightarrow$) Let $w \in$ Cov. For some i, $a \in A_i$, $b \in B_i$, $k \ge 0$ we have $w \sqsubseteq ab^k$. Since $w \notin \mathrm{pr}(A_i)$, w is not a prefix of a. Hence a is a proper prefix of w. Put $j = |a|$. Then $w[:j] = a \in A_i$, and (a) holds. Next, if $b = \varepsilon$, then $ab^k = a$, which contradicts $w \sqsubseteq a$. So $b \ne \varepsilon$. Since $w \notin A_i \,\mathrm{pr}(B_i)$, $w[j:]$ is not a prefix of b. On the other hand $w[j:] \sqsubseteq b^k$, so $|w[j:]| > |b|$. Put $t = j + |b|$. Then $j < t < |w|$ and $w[j:t] = b \in B_i$, so (b) holds. Finally, $w[j:]$ is a prefix of b^ω. Since $w[t:]$ is $w[j:]$ shifted by $|b|$ letters, it is also a prefix of b^ω. Of two prefixes of the same infinite word, the shorter is a prefix of the longer, so $w[t:] \sqsubseteq w[j:]$, and (c) holds. □

Corollary 5.6. *$E = \Sigma^*$ if and only if every word of* Cand $\setminus$ Mid *has a cut.*

Proof. ($\Leftarrow$) The words of Cand $\setminus$ Mid have a cut, so by the direction ($\Leftarrow$) of Lemma 5.5 they belong to Cov. The words of Cand $\cap$ Mid belong to Cov by Lemma 5.3 (2). Hence Cand $\subseteq$ Cov, and by Lemma 5.3 (1), $E = \Sigma^*$.

($\Rightarrow$) If $E = \Sigma^*$, then by Lemma 5.3 (1), Cand $\subseteq$ Cov. A word $w \in$ Cand $\setminus$ Mid belongs to Cov, and $w \notin$ Mid, so by the direction ($\Rightarrow$) of Lemma 5.5 it has a cut. □

In the direction ($\Leftarrow$) of the corollary we use Mid $\subseteq$ Cov, and in ($\Rightarrow$) we use the direction ($\Rightarrow$) of Lemma 5.5 under $w \notin$ Mid. In the direction ($\Rightarrow$) of Lemma 5.5, the assumption $w \notin$ Mid cannot be dropped. For example, in Example 5.7 below, $w = 0^m1$ is a prefix of 0^m10, so it belongs to Cov. However, w does not contain the factor 10, so it has no cut. If we decided by the existence of a cut without excluding Mid, we would wrongly classify this word as an avoider.

Example 5.7. Let $\Sigma = \{0, 1\}$, and take one cap component, $A = 0^*$, $B = \{10\}$. Let also $C = 0^*1\Sigma^3$. That is, $C\Sigma^*$ is the set of "words in which 1 is followed by at least 3 letters". Then

$$\text{Cand} = 0^* \cup 0^*1\Sigma^{\le 2}, \qquad \text{Mid} = 0^* \cup 0^*1 \cup 0^*10$$

and Cand $\setminus$ Mid consists of the words 0^m11, 0^m100, 0^m101, 0^m110, 0^m111 ($m \ge 0$). The anchor condition $w[:j] \in 0^*$ is satisfied by j only with $j \le m$. Among these, $w[j:j{+}2] = 10$ holds only for $j = m$, and this happens only when w starts with 0^m10. If $w = 0^m100$, the tail is $w[m{+}2:] = 0$, which is not a prefix of $w[m:] = 100$. If $w = 0^m101$, the tail is 1, which is a prefix of 101. Hence only 0^m101 has a cut, and that cut is $(m, m{+}2)$. By Corollary 5.6, E is not universal, and for example 11 is an avoider.

Remark 5.8. By Lemma 5.5, whether an individual word belongs to Cov can be decided by examining finitely many pairs of positions (j, t). However, Cand $\setminus$ Mid is in general an infinite set, and the covered language Cov is in general not necessarily regular. For example, let $\Sigma = \{0, 1\}$, $A = \{\varepsilon\}$, $B = 0^*1$, and put $L =$ Cov $= \mathrm{pr}(B^\circ)$. Then

$$L \cap 0^*10^*1 = \{\, 0^n10^n1 \mid n \ge 0 \,\}$$

holds. Indeed, $0^n10^m1 \notin$ Mid, and the anchor is empty, so $j = 0$. The root belongs to 0^*1 and the tail is nonempty, so the root must be the part up to the first 1, namely 0^n1. The remaining tail 0^m1 is a prefix of 0^n10^m1 only when $m = n$. The right-hand side is nonregular (for different n, the prefixes 0^n1 are distinguished by the continuation 0^n1). Regular languages are closed under intersection, so L is also nonregular. Hence we cannot construct a general Cov as a DFA and reduce the problem directly to an inclusion test with Cand. Reducing the comparison in the border condition to a finite computation is the purpose of the following subsections and of Section 6.

5.2 The live part

In this subsection we show that for the words of Cand that can be extended infinitely within Cand (live words), the coverage test reduces to one regular language inclusion test. We use the notation of Section 5.1.

5.2.1 Two constants

We fix one complete DFA accepting Cand. For the state reached from the initial state by reading a word u we write $\delta(u)$, and for the state reached from a state q by reading a word u we write $\delta(q, u)$. Unreachable states are removed in advance. Since Cand is prefix-closed, no accepting state can be reached from a reachable non-accepting state. Hence we may merge the non-accepting states into one sink. From now on we always consider a DFA of this form. Also, for an infinite word y, $\mathrm{pr}(y)$ denotes the set of finite prefixes of y.

Definition 5.9 (n_{acc} and n_{root})**.** Let n_{acc} be the number of accepting states of the complete DFA of Cand fixed above (the sink is not counted). For each i, let the state set of a DFA accepting B_i be Q_i, and let $n_{\mathrm{root}} = \max(\{1\} \cup \{|Q_i| \mid i \in I\})$.

Both can be computed from the input DFAs. If there is no component at all, then $n_{\mathrm{root}} = 1$.

5.2.2 Rays

Definition 5.10 (Ray)**.** Let $w \notin \mathrm{Mid}$. For a component i and a cut of w with respect to it, say (j, t), put $r = w[j{:}t]$, and call the infinite word $w[{:}j]\, r^{\omega}$ a ray of w.

By the definition of a cut, $t - j \geq 1$, so r is nonempty and r^{ω} is defined. Also, by the border condition, $w[j{:}]$ is a prefix of r^{ω}, so w is a prefix of each of its rays. Since w has only finitely many cuts, w also has only finitely many rays. A ray represents the extension that "keeps repeating a root already visible in w".

Lemma 5.11 (Covered extensions lie on rays)**.** *Let $w \notin \mathrm{Mid}$, and let w' be an extension of w ($w \sqsubseteq w'$). Then $w' \in \mathrm{Cov}$ if and only if w' is a prefix of some ray of w.*

Proof. ($\Leftarrow$) Let $w' \sqsubseteq w[{:}j]\, r^{\omega}$. Here $w[{:}j] \in A_i$ and $r \in B_i$. For sufficiently large k we have $w' \sqsubseteq w[{:}j]\, r^{k}$, and $w[{:}j]\, r^{k} \in A_i B_i^{\circ}$, so $w' \in \mathrm{pr}(A_i B_i^{\circ}) \subseteq \mathrm{Cov}$.

($\Rightarrow$) Since Mid is prefix-closed and $w \notin \mathrm{Mid}$, we have $w' \notin \mathrm{Mid}$. Hence, by Lemma 5.5 of Section 5.1, w' has, with respect to some component i, a cut (j, t). Suppose $t \geq |w|$. Then w is a prefix of $w'[{:}t] = w'[{:}j]\, w'[j{:}t] \in A_i B_i$. Prefixes of words of $A_i B_i$ belong to $\mathrm{pr}(A_i) \cup A_i\, \mathrm{pr}(B_i) \subseteq \mathrm{Mid}$, so $w \in \mathrm{Mid}$, a contradiction. Hence $t < |w|$. Then $w[{:}j] = w'[{:}j] \in A_i$ and $w[j{:}t] = w'[j{:}t] \in B_i$. Also, $w'[j{:}]$ has period $t - j$, so its prefix $w[j{:}]$ also has period $t - j$. Hence (j, t) is a cut of w, and with $r = w[j{:}t]$, w' is a prefix of the ray $w[{:}j]\, r^{\omega}$. □

5.2.3 Live words and ShortCov

Definition 5.12 (Live)**.** A word $w \in \mathrm{Cand}$ is live if there exists an infinite word x such that all finite prefixes of wx belong to Cand. The set of live words is written Live. Words of Cand that are not live are called nonlive.

In the DFA of Cand, an accepting state q is said to be live if there is an infinite path from q that passes through accepting states only. A word $w \in \mathrm{Cand}$ is live if and only if $\delta(w)$ is a live state. Hence Live is a regular language, and its DFA is computable. Prefixes of live words, and extensions of live words that stay in live states, are both live.

Definition 5.13 (ShortCov)**.**

$$\mathrm{ShortCov} \;=\; \mathrm{Mid} \;\cup \bigcup_{i \in I} \bigcup_{\substack{r \in B_i \\ 1 \leq |r| \leq n_{\mathrm{acc}} n_{\mathrm{root}}}} A_i\, \mathrm{pr}(r^{\omega}).$$

The inner union is over finitely many r, and $\mathrm{pr}(r^{\omega})$ is regular. Hence ShortCov is a regular language, and its DFA is computable. Moreover, $\mathrm{ShortCov} \subseteq \mathrm{Cov}$. Indeed, $\mathrm{Mid} \subseteq \mathrm{Cov}$, and if $a \in A_i$, $r \in B_i$, $u \sqsubseteq r^{\omega}$, then for some k we have $u \sqsubseteq r^{k}$, so $au \in A_i B_i^{\circ}$.

5.2.4 Live lemma

Lemma 5.14 (Live lemma)**.** $\mathrm{Live} \subseteq \mathrm{Cov}$ *if and only if* $\mathrm{Live} \subseteq \mathrm{ShortCov}$.

Proof. ($\Leftarrow$) follows from ShortCov $\subseteq$ Cov.

($\Rightarrow$) Assume Live $\subseteq$ Cov. Take $w \in$ Live. If $w \in$ Mid, then $w \in$ ShortCov, so let $w \notin$ Mid. We divide the proof into four steps.

(1) An extension is one of finitely many rays. Let x be an infinite word such that all finite prefixes of wx belong to Cand. All finite prefixes of wx are live, so by assumption they belong to Cov. By Lemma 5.11, those that extend w among the finite prefixes of wx are each a prefix of some ray of w. There are finitely many rays, so a single ray shares a prefix with arbitrarily long prefixes of wx. Hence wx equals that ray. In particular, there are only finitely many such extensions wx.

(2) An extension follows a short cycle. Consider the subgraph of the DFA of Cand consisting of the live states only. Suppose that a state reachable from $\delta(w)$, say p, lies on a cycle of this subgraph. The only edge from p to a live state is the edge of the cycle. Indeed, suppose there is another such edge, by a letter b, and let u be the word read once around the cycle. Let a word leading from $\delta(w)$ to p be v. Then for $k = 0, 1, 2, \ldots$ we obtain an infinite extension starting with $wvu^k b$. These differ from one another in whether the letter right after u^k is b or the first letter of u, so they are infinitely many distinct extensions, which contradicts (1). Now let wx be an extension as in (1). The infinite path from $\delta(w)$ reading x passes through some state p infinitely often. The path from p is uniquely determined by the property above, so the path repeats the closed path that leaves p and first returns to p. On this closed path no state other than p appears twice (if one did, the path uniquely determined from that state would not return to p). Hence the length of the closed path, e, is at most the number of accepting states, that is, $e \le n_{\mathrm{acc}}$, and wx has period e from a sufficiently distant position on.

(3) The root is a power of a short nonempty word. By (1) we can write $wx = w[:j]\, r^\omega$. Here $(j, j+p)$ is, with respect to component i, a cut of w, and $r = w[j{:}j{+}p] \in B_i$. At a position $n \ge j$ we have $(wx)[n] = r[(n-j) \bmod p]$, and for sufficiently large n we have $(wx)[n] = (wx)[n+e]$. Comparing all residue classes at sufficiently distant positions, we see that r does not change under a cyclic shift by e letters. Put $g = \gcd(p, e)$. By Bézout's identity we have $g = ue + vp$ for some integers u, v. A cyclic shift by length p does not change r, so performing the shift by e letters u times (in the opposite direction if negative) gives a shift by g letters. Hence r does not change under a cyclic shift by g letters. For example, if $p = 6$, $e = 4$, then $g = 2$, and in r the even-numbered letters are all equal and the odd-numbered letters are all equal ($r = \pi^3$, $|\pi| = 2$). Since g divides p, with $\pi = r[:g]$ we have $r = \pi^{p/g}$. Here $|\pi| = g \le e \le n_{\mathrm{acc}}$.

(4) A short root can be taken. In the DFA of B_i, let reading π^n from the initial state lead to the state s_n ($n \ge 0$). Since $r = \pi^{p/g} \in B_i$, the state s_n is accepting for some $n \ge 1$. Let n_0 be the least such number. Suppose $n_0 > |Q_i|$. Then among $s_0, \ldots, s_{n_0-1}$ there is a pair with $s_a = s_b$ ($a < b$). Then $s_{n_0-(b-a)} = s_{n_0}$ is accepting, and $n_0 - (b-a) \ge a + 1 \ge 1$, which contradicts the minimality of n_0. Hence $n_0 \le |Q_i| \le n_{\mathrm{root}}$. Put $r' = \pi^{n_0}$. Then $r' \in B_i$ and $1 \le |r'| = n_0 g \le n_{\mathrm{acc}} n_{\mathrm{root}}$. Moreover $r'^\omega = \pi^\omega = r^\omega$, so $w[j:]$ is a prefix of r'^ω. Since $w[:j] \in A_i$, we have $w \in A_i \operatorname{pr}(r'^\omega) \subseteq$ ShortCov. □

Corollary 5.15. *1. If* Live $\not\subseteq$ ShortCov*, then E is not universal.*

2. If Live $\subseteq$ ShortCov*, then E is universal if and only if every word of* Dead $=$ Cand $\setminus$ (Live $\cup$ Mid) *has a cut.*

Proof. (1) By Lemma 5.14, Live $\not\subseteq$ Cov, and since Live $\subseteq$ Cand, we have Cand $\not\subseteq$ Cov. (2) By Lemma 5.14, Live $\subseteq$ Cov, and also Mid $\subseteq$ Cov. Hence Cand $\subseteq$ Cov is equivalent to Dead $\subseteq$ Cov. Words of Dead do not belong to Mid, so by Lemma 5.5 of Section 5.1, this is equivalent to every word of Dead having a cut. □

Remark 5.16. In case (1), the word of Live $\setminus$ ShortCov returned by the inclusion test is not necessarily itself an avoider. That word may be covered by a long root. What Lemma 5.14 guarantees is only that an avoider exists somewhere in Live. At this point the procedure returns only the truth value of "an avoider exists". An actual avoider can be obtained after the decision by examining words in order of length (Corollary 6.26).

Example 5.17. In Example 5.7 of Section 5.1, we choose as the complete DFA of Cand one whose accepting states are four states, the state that keeps reading 0 and the states that, after 1, have read $0, 1, 2$ letters (together with the sink). Then $n_{\mathrm{acc}} = 4$. Only the first state has an infinite path through accepting states only, so Live $= 0^*$. Since $0^* \subseteq$ Mid $\subseteq$ ShortCov, the inclusion holds, and the decision moves to the problem on Dead $= 0^*\{11, 100, 101, 110, 111\}$. On the other hand, if we make C empty, then Cand $=$ Live $= \Sigma^*$, and since $11 \notin$ ShortCov, by Corollary 5.15 (1), E is not universal. Indeed 11 is an avoider.

Remark 5.18. Under the assumption Live $\subseteq$ Cov, the infinite extensions of words of Live $\setminus$ Mid within Cand are eventually periodic by (1)–(3). Therefore, on the live part there is no need to consider the nesting depth of border conditions. This argument is not available on the nonlive part. Instead, we show in the next subsection that the nesting depth is bounded.

5.3 The rank of the nonlive part

By the previous subsection, if Live $\subseteq$ ShortCov holds, then avoiders exist only in Dead = Cand $\setminus$ (Live $\cup$ Mid). Words of Dead have no infinite extension within Cand, so the periodicity argument of the previous subsection does not apply. In this subsection we show instead that the nesting depth of border conditions is bounded. This bound determines the depth of the recursion from Section 6.3 on.

5.3.1 Rank

Definition 5.19 (Rank)**.** Borders and nontrivial borders were defined in Section 3. For a word w, the maximum, over all nonempty suffixes of w (written z), of "the number of nontrivial borders of z" is written $\mathrm{rank}(w)$. We set $\mathrm{rank}(\varepsilon) = 0$.

The rank can be restated as a way of counting cuts, as follows. However, what we count here are pairs of positions that ignore the anchor and root conditions and satisfy only the border condition. Fix a left position j ($0 \le j < |w|$), and consider the condition $j < t < |w|$ and $w[t{:}] \sqsubseteq w[j{:}]$. The set of right positions t satisfying it is written T_j. Since $w[t{:}]$ is always a suffix of $w[j{:}]$, $t \in T_j$ is equivalent to $w[t{:}]$ being a nontrivial border of $w[j{:}]$ (Section 5.1). The map $t \mapsto w[t{:}]$ is a bijection from T_j to the set of nontrivial borders of $w[j{:}]$ (its inverse sends a border of length ℓ to $t = |w| - \ell$). As j varies, $w[j{:}]$ ranges over all nonempty suffixes of w, so

$$\mathrm{rank}(w) = \max_{0 \le j < |w|} |T_j|$$

holds (when $w \neq \varepsilon$). That is, $\mathrm{rank}(w)$ is the maximum number of right positions satisfying the border condition for a single left position. If the borders of one word are arranged by length, the shorter ones are also borders of the longer ones, so these right positions are nested. This is why $\mathrm{rank}(w)$ is called the nesting depth of border conditions. The anchor and root conditions are not counted, so this is a quantity of w alone, independent of the cap expression.

5.3.2 The rank lemma

The proof uses the following fact.

Lemma 5.20. *Let μ be a nonempty word and ν a word. If $\nu \sqsubseteq \mu\nu$, then $\mu\nu$ is a prefix of μ^ω.*

Proof. Put $x = \mu\nu$. From $\nu \sqsubseteq x$, for $0 \le k < |\nu|$ we have $x[|\mu| + k] = \nu[k] = x[k]$. That is, x has period $|\mu|$. Since $x[{:}|\mu|] = \mu$, x is a prefix of μ^ω. □

This is the same fact as the one used in the proof of Lemma 5.5 of Section 5.1.

Lemma 5.21 (Rank lemma)**.** *If $w \in$ Cand $\setminus$ Live, then $\mathrm{rank}(w) \le h$. Here $h = \max(0,\ n_{\mathrm{acc}} - 2)$.*

Proof. If $w = \varepsilon$, then $\mathrm{rank}(w) = 0 \le h$. Let $w \neq \varepsilon$. Let z be a nonempty suffix of w, and let the lengths of the nontrivial borders of z, in decreasing order, be $\ell_1 > \cdots > \ell_r$. Put $\ell_0 = |z|$, $\ell_{r+1} = 0$, and let $k_i = |w| - \ell_i$ ($0 \le i \le r+1$). We have $k_0 < k_1 < \cdots < k_{r+1} = |w|$. Since Cand is prefix-closed, $w[{:}k_i] \in$ Cand, and the states $q_i = \delta(w[{:}k_i])$ are all accepting. (We add two, for z itself and for the empty word, so the positions number not r but $r + 2$. A concrete example is given after the proof.)

We show that the $r+2$ states $q_0, \ldots, q_{r+1}$ are pairwise distinct. Suppose $q_i = q_l$ ($i < l$), and put $\mu = w[k_i{:}k_l]$, $\nu = w[k_l{:}]$. Since $k_i < k_l$, μ is nonempty. The word $w[k_i{:}] = \mu\nu$ is the border of z of length ℓ_i (if $i = 0$, z itself), and ν is the border of z of length ℓ_l. ν is a prefix of z, and $\mu\nu$ is also a prefix of z and longer than ν, so $\nu \sqsubseteq \mu\nu$. By Lemma 5.20, $\mu\nu \sqsubseteq \mu^\omega$, and hence $w = w[{:}k_i]\,\mu\nu$ is a prefix of the infinite word $y = w[{:}k_i]\,\mu^\omega$. Since $\delta(q_i, \mu) = q_l = q_i$, μ is a loop returning to the state q_i. The intermediate states of this loop are $\delta(w[{:}k])$ ($k_i \le k \le k_l$), all of which are accepting. Hence all finite prefixes of y belong to Cand. Since w is a prefix of y, w is live. This contradicts $w \notin$ Live.

Hence $q_0, \ldots, q_{r+1}$ are $r + 2$ pairwise distinct accepting states, and $r + 2 \le n_{\mathrm{acc}}$. Since z was arbitrary, $\mathrm{rank}(w) \le n_{\mathrm{acc}} - 2 \le h$. □

Intuitively, too many borders make states of the DFA repeat, which creates a periodic loop and allows w to be extended infinitely.

Words of Dead belong to Cand $\setminus$ Live, so below we use the fact that the rank of every word of Dead is at most h.

Example 5.22. For the DFA of Cand chosen in the example of the previous subsection, $n_{\mathrm{acc}} = 4$, so the upper bound is 2. Since 0^m100 and 111 belong to Cand$\setminus$Live, their rank is at most 2. When $m \ge 2$, $w = 0^m100$ has the nontrivial borders 0 and 00, so $\mathrm{rank}(w) = 2$. Also $w = 111$ has the nontrivial borders 1 and 11, so $\mathrm{rank}(w) = 2$. Hence the upper bound of the rank lemma is attained. On the other hand, when $m \le 1$, no nonempty suffix of

0^m100 has two or more nontrivial borders, and the suffix 00 has one, so the rank is 1. Following the states of the proof for $w = 00100$, the border lengths of $z = w$ are $5 > 2 > 1 > 0$, the positions are $k = 0, 3, 4, 5$, and the states reached by the prefixes $\varepsilon, 001, 0010, 00100$ are pairwise distinct and exhaust the four accepting states.

Remark 5.23. The rank lemma is a statement about nonlive words. The rank of live words is in general unbounded (for example, with $\Sigma = \{0, 1\}$ and no C_j, for $n \geq 1$ we have $\mathrm{rank}(0^n) = n - 1$). This is why the live part is handled first by the periodicity argument of the previous subsection.

6 Deciding the nonlive part: encoding and lowering

We review the development so far. We called a word that does not belong to a restricted cap set expression E an avoider (Definition 5.1). E is universal exactly when no avoider exists. In Section 5 we showed

$$E = \Sigma^* \iff \text{Live} \subseteq \text{ShortCov} \ \wedge \ \big(\text{Dead consists of words that have a cut}\big)$$

and saw that the first conjunct can be decided by an inclusion test for regular languages. What remains is the second conjunct. A word of Dead belongs to Cand and not to Mid, so a word of Dead without a cut belongs neither to Cov nor to $\bigcup_j C_j\Sigma^*$, and is an avoider (Lemma 5.5). Conversely, under Live $\subseteq$ ShortCov, the words of Mid and Live are covered, so avoiders occur only in Dead. Hence what we must decide is "does Dead contain a word without a cut, that is, an avoider?" The cut condition contains the border condition, and Cov is not regular in general, so we cannot construct Cov as a DFA and test inclusion (Remark 5.8). In this section we use the fact that the words of Dead have rank at most h (rank lemma, Lemma 5.21), peel off the nesting of the border condition one layer at a time by block encoding, and finally reduce, by lowering, to the evaluation of a first-order sentence over a finite alphabet.

Section 6.1 gives an overview of this whole section. Section 6.2 formulates the problem in a general form (the avoidance problem) and solves the case in which no firing with a nonempty tail occurs. Section 6.3 lowers the rank by one by block encoding. Section 6.4 translates, by lowering, sentences of the upper level into formulas of the lower level, and assembles the whole procedure and the proof of the main theorem.

6.1 Overview of this section

Summary of the approach. We restate here the approach described in Section 2 (the treatment of Dead). We take as a candidate word the word obtained from a word w of Dead by prepending a letter σ that serves to match the form, namely σw. We call the first letter the seed and the remaining w the body. We rewrite the cut condition of each component of the cap expression in the following form: a pair (row) of an automaton that reads words with markers and a condition (guard) between the letters just before the two cut points fires. At the first stage the guard is always true, and the candidate words on which no row fires correspond exactly to the avoiders.

The condition for firing contains the border condition, so automata cannot handle it as it stands. We therefore divide the body w at its last letter a into blocks, $w = U_1\, a\, U_2\, a \cdots U_m\, a$, and, treating each a-free block U_i as one letter, we form the word $U_1\,(U_2, \ldots, U_m)$ (block encoding). The first block U_1 becomes the new seed, and the remaining sequence becomes the new body. For example, if $a = 0$ and $w = 01010$, then $U_1 = \varepsilon$, $U_2 = U_3 = 1$, and the encoding is $[\,]\,([1], [1])$. The border condition then splits into a condition between the two blocks containing the cut points (which goes into the new guard) and the equality of the block sequences after them (the border condition of the new word), and the nesting of the latter is one layer shallower. In the example, the rank of w is 2 (the suffix 01010 has the nontrivial borders 0 and 010), and the rank of the new body $([1], [1])$ is 1.

For a firing whose right cut point is at the end of the word (a firing whose tail is empty), the border condition is trivial, so such a firing can be represented by an automaton and subtracted from the candidate at each stage. By the rank lemma, the words of Dead have rank at most h, so after at most h repetitions of block encoding, no firing with a nonempty tail remains. Each encoding turns "one word of the previous stage" into "one letter of the next stage", so the problem at each stage is a problem on a structure whose letters are the words of the previous stage. Finally, lowering translates the first-order sentence of each stage into a formula of the stage below, one stage at a time, and returns to a sentence over the original finite alphabet.

About the tables. The tables below summarize in advance the relations among objects that are formally defined in later sections. On a first reading it is enough to glance at the tables; they are meant to be consulted again while reading each section.

Objects at each level. From a problem at level k, one application of block encoding yields a problem at level $k+1$. Below, we write the objects of the level before encoding without a subscript, and the objects of the level after encoding with a prime.

	Before encoding (level k)	After encoding (level $k+1$)
Structure	M (domain D)	$M' = M'[s, a, \bar{p}]$ (domain $D' = D^*$)
One letter	an element of D	one word over D (a block)
Candidate	$\mathcal{C}$	$\mathcal{C}'$ (the encodings of the words of $\mathcal{C}^0_{s,a}$)
Row	(T, G)	(T', H)
What the guard compares	$\mathrm{pred}(j), \mathrm{pred}(t)$ (two letters of D)	U_i, U_l (two blocks)
Position of the cut	(j, t)	$(i-1, l-1)$ (the indices of the blocks containing j, t)
Border condition	the whole	the remaining part (one layer shallower)
Parameters	$\bar{p}$	$\bar{p}$ and the slice parameters of this stage $s, a \in D$

In particular, "one word of the lower level" becomes "one letter of the upper level". To distinguish letters, in the examples we write the letters of level 1 (blocks) in square brackets, as in $[0^n]$, $[0]$, $[\,]$ (the empty block), and the letters of level 2 (sequences of blocks) in round brackets, as in $(\,)$ (the empty sequence), $([\,])$.

Names and contents of predicates. $\mathrm{Run}_{q,r}$ and H are relation symbols of the signature of M', not predicates of M (their arguments are not elements of D but elements of D^*). While we work on M' (T', $\mathcal{C}'$, and the assembly of formulas at the next level), these appear only by name, as atomic formulas of M', $\mathrm{Run}_{q,r}(x)$ and $H(x, y)$, and their contents are not consulted. For example, in T' the edge label $\mathrm{Run}_{q,r}(x)$ is, in the same sense in which a formula of $(\mathbb{N}; <)$ contains the atomic formula $x < z$, a formula of M'. The meaning of these symbols (for which elements they are true) is given by automata over M. One argument of a predicate is an element of D^*, that is, a word over D, and it is the whole input written on one tape of the automaton (for the binary H there are two tapes). Hence acceptance by a symbolic automaton over M' is, semantically, nested: the outer automaton reads one block per step, and the interpretation of a predicate on that block is given by whether the inner automaton over M accepts the sequence of letters of the block. This does not mean that truth values of predicates are evaluated here. Lowering (Section 6.4) is the operation that uses this presentation, following the construction of a sentence of M', to assemble an automaton over M, and that expresses its nonemptiness by a formula of M.

Treatment of the parameters s, a. The slice parameters s, a change roles in the following order.

Step	Status of s, a
Building the slice, $F_{s,a}$, $\mathcal{C}^0_{s,a}$ (Section 6.2)	free variables of formulas and automata of M (one value assumed)
Building $M'[s, a, \bar{p}]$ and treating the next level on it	part of the interpretation of M' (not free variables of formulas of M')
Applying lowering to the M'-sentence Φ' obtained at the next level	surface as free variables of $\mathrm{Down}(\Phi')(s, a, \bar{p})$
Building $\Phi = \mathrm{SeedOnly} \vee \exists s\, \exists a.\, \mathrm{Down}(\Phi')$	bound by quantifiers of M

The values of s, a are not enumerated; they pass through the construction once, as symbols. Hence there is no recursion on each of the infinitely many slices.

The following table lists the case of two stacked stages, with the values of the example in Section 7 ($D_0 = \Sigma$, $D_1 = \Sigma^*$, $D_2 = D_1^*$). The subscript of a formula (the lower index in $\Phi_2, \Phi'_1, \Phi''_0$) indicates the remaining number of recursive steps (the upper bound on the rank), and the number of primes indicates the encoding stage. By contrast, the subscripts on M, D, Down and SeedOnly denote the level.

Formula	Structure of the formula	Free variables	Parameters in the interpretation
Φ''_0	$M_2 = M'_1[s_1, a_1]$	none (sentence)	s_0, a_0, s_1, a_1
$\mathrm{Down}_1(\Phi''_0)(s_1, a_1)$	$M_1 = M'_0[s_0, a_0]$	$s_1, a_1 \in D_1$	s_0, a_0
$\Phi'_1 = \mathrm{SeedOnly}_1 \vee \exists s_1 \exists a_1. \cdots$	M_1	none (sentence)	s_0, a_0
$\mathrm{Down}_0(\Phi'_1)(s_0, a_0)$	$M_0 = \Sigma$	$s_0, a_0 \in D_0$	none
$\Phi_2 = \mathrm{SeedOnly}_0 \vee \exists s_0 \exists a_0. \cdots$	Σ	none (sentence)	none

In the example, $s_0 = \sigma$, $a_0 = 1 \in D_0$, $s_1 = [0^n]$, $a_1 = [\,] \in D_1$ are the values on the branch that is true. In Φ'_1, the quantifier $\exists a_1$ ranges over elements of D_1 (words over Σ), so when Down_0 translates Φ'_1, a_1 becomes one tape and is finally removed by projection.

Shape of the procedure. What the procedure assembles can be written as the following recursion (formally, Theorem 6.24).

> Build$(M, \mathcal{R}, \mathcal{C}, h)$:
> With s, a as new symbols, build $F_{s,a}$ and $\mathcal{C}^0_{s,a}$ (Section 6.2).
> If $h = 0$, return SeedOnly $\vee\, \exists s\, \exists a.$ NonEmpty$_{\mathcal{C}^0_{s,a}}(s, a)$.
> Otherwise, build the presentation of $M'[s, a, \bar{p}]$ (the automata giving the predicates), $\mathcal{R}'$ and $\mathcal{C}'$ (Section 6.3), set $\Phi' :=$ Build$(M', \mathcal{R}', \mathcal{C}', h-1)$, and return SeedOnly $\vee\, \exists s\, \exists a.$ Down$(\Phi')(s, a)$.

There is exactly one recursive call at each stage, and what is returned is always a finite formula. The truth value of a formula is evaluated only once, at the end, on returning to level 0 (Σ).

What is computed, and what is used only in proofs.

What the procedure builds as syntax or evaluates	What is used only in the correctness proof
Cand, Live, ShortCov and the inclusion test (Section 5.2)	cuts and rays (Sections 5.1 and 5.2)
n_{acc} and the recursion depth $h = \max(0, n_{\mathrm{acc}} - 2)$	the fact that the body of each candidate has rank at most h (rank lemma, strict descent)
the symbolic automata of $F_{s,a}$, $\mathcal{C}^0_{s,a}$	
the automata giving Run, H and so on, $\mathcal{R}'$, $\mathcal{C}'$	the concrete word decomposition $\mathrm{enc}_{s,a}$ and the correspondence of positions
the formulas Φ and Down(Φ), and the final evaluation over Σ	the firing position j (it goes into the nondeterminism of the automaton)

In particular, the rank condition is not something the procedure checks. As a hypothesis of Theorems 6.8 and 6.24, it is guaranteed by the rank lemma and strict descent.

6.2 Rows and firing

We formulate the question of whether Dead contains an avoider as a general problem whose form does not change under repeated encoding. In this section we define that problem and solve the case in which no firing with a nonempty tail occurs (the case of rank 0). In this section and the next, we write the structure, alphabet and row of the level before encoding without subscripts as $M, D, (T, G)$, and those after encoding as $M', D', (T', H)$.

6.2.1 Candidates, rows, firing

Fix a structure M (with domain D) and a tuple of parameters $\bar{p}$. The markers L, R are two letters that do not belong to D.

Definition 6.1 (Candidate, seed, body)**.** A candidate is, with parameters $\bar{p}$, an M-regular language $\mathcal{C} \subseteq D^+$. We write a candidate word as $s\,w$ ($s \in D$, $w \in D^*$), and we call s the seed and w the body. For a position k of the body ($0 \le k \le |w|$), we write the letter just before it as $\mathrm{pred}(k)$. That is, $\mathrm{pred}(0) = s$, and for $1 \le k \le |w|$, $\mathrm{pred}(k) = w[k-1]$.

Definition 6.2 (Row and firing)**.** A row is a pair (T, G). Here T is a symbolic automaton over $D \cup \{L, R\}$ (with parameters $\bar{p}$), and $G(x, y; \bar{p})$ is a formula of M (the guard). We say that the row (T, G) fires on the candidate word $s\,w$ at positions (j, t), $0 \le j < t \le |w|$, if all of the following three conditions hold.

(i) T accepts $s\,w[:j]\,L\,w[j:t]\,R\,w[t:]$.

(ii) $M \models G(\mathrm{pred}(j), \mathrm{pred}(t); \bar{p})$.

(iii) $w[t:] \sqsubseteq w[j:]$ (border condition).

When $t = |w|$ we call this a firing with an empty tail, and when $t < |w|$ a firing with a nonempty tail.

The $\sqsubseteq$ in (iii) compares by equality of letters of D. In a firing with an empty tail, $w[t:] = \varepsilon$, so (iii) holds trivially. The guard is evaluated only on one pair of letters, namely the letters just before the two cut points.

Definition 6.3 (Avoidance problem)**.** For a finite set $\mathcal{R}$ of rows and a candidate $\mathcal{C}$, the problem asking "is there a word of $\mathcal{C}$ on which no row of $\mathcal{R}$ fires at any position?" is called the avoidance problem $(M, \mathcal{R}, \mathcal{C})$. Such a word is called a solution of this avoidance problem.

6.2.2 Correspondence with the avoiders of Dead

Assume that Σ is nonempty, and fix a letter $\sigma \in \Sigma$. Let $M = \Sigma$, and let the candidate be $\sigma\,\mathrm{Dead} = \{\sigma w \mid w \in \mathrm{Dead}\}$. This is a regular language. For each cap component i, we define a row $(T_i, \top)$ as follows. T_i is an NFA that skips the first letter (the seed) and checks the following before L, between L and R, and after R, respectively: the part before L belongs to A_i, the part between L and R belongs to B_i, and the part after R is nonempty. The guard is the always-true formula $\top$ (here $\top$ is logical truth, and is different from the tree label T). We call this a cap row.

Lemma 6.4. *Let $w \in \mathrm{Dead}$. The cap row $(T_i, \top)$ fires on σw at positions (j,t) if and only if (j,t) is, for component i, a cut of w. Consequently, no cap row fires on the candidate word σw if and only if w is an avoider.*

Proof. T_i accepts $\sigma\, w[:j]\, L\, w[j{:}t]\, R\, w[t:]$ if and only if $w[:j] \in A_i$, $w[j{:}t] \in B_i$ and $t < |w|$. The guard always holds. Condition (iii) is exactly the border condition of a cut. This proves the first part. The second part follows from $w \notin \mathrm{Mid}$, from Section 5.1 (Lemma 5.5), and from $w \in \mathrm{Cand}$. □

At the bottom stage, the seed σ is a letter that serves only to match the form. T_i skips the seed and the guard is always true, so the choice of σ does not affect the result. The seed becomes meaningful only after the encoding of Section 6.3.

6.2.3 Slices and firing with an empty tail

The guard (ii) compares letters at two distant places in the word. When D is infinite, a finite-state automaton cannot remember a letter read midway until the end of the word. We therefore introduce parameters as follows.

Definition 6.5 (Slice)**.** For $s, a \in D$, the set of candidate words whose seed is s and whose body is nonempty with last letter a is called the (s,a) slice of the candidate.

A candidate word either has an empty body or belongs to exactly one slice. The (s,a) slice is the intersection of the candidate with the language of "words of length at least 2 whose first letter is s and whose last letter is a". The latter language is accepted as follows: put the label $x = s$ on the first column and the label $x = a$ on the last column (guessed nondeterministically from the second column on); this gives a symbolic automaton (with parameters s, a). Hence the (s,a) slice is, with parameters $s, a, \bar{p}$, an M-regular language (Lemma 3.2).

Lemma 6.6 (Firing with an empty tail is regular)**.** *Let s, a be parameters. Among the words of the (s,a) slice, let those on which some row of $\mathcal{R}$ fires with an empty tail form the set $F_{s,a}$. It is, with parameters $s, a, \bar{p}$, an M-regular language, and its symbolic automaton can be built from syntax alone.*

Proof. It suffices to build one for each row and take the union (Lemma 3.2). Fix a row (T, G). In a firing with an empty tail, $t = |w|$, so $\mathrm{pred}(t) = a$, and (ii) becomes $G(\mathrm{pred}(j), a; \bar{p})$. This is a condition on $\mathrm{pred}(j)$ only. We build the following symbolic automaton.

1. Read the seed and imitate the transitions of T so that the initial condition of T and the condition of the edge reading the seed hold.
2. Read the body letter by letter while imitating the transitions of T (the labels of T are used as they are).
3. Choose a position j nondeterministically. If $j = 0$, impose the parameter-only condition $G(s, a; \bar{p})$. If $j \geq 1$, on the label of the edge reading the letter at position $j-1$, add $G(x, a; \bar{p})$ with $\wedge$. Then, in T, follow one L edge.
4. Read the rest of the body. However, do not proceed to the next phase until at least one letter of the body has been read after L (this guarantees $j < t$).
5. Finally, in T, follow an R edge and end in an accepting state of T.

The marker edges of T read no input, so at this point they become moves that consume no letter. Their conditions are parameter-only formulas, so we absorb them by adding them with $\wedge$ to the label of an adjacent data edge (or to the initial or acceptance condition). What we obtain is a symbolic automaton over M in the sense of Definition 3.1. Its accepting paths correspond one-to-one to a choice, subject to $j < t = |w|$, of a position j together with the validity of (i) and (ii) there. Taking the product of this with the symbolic automaton of the (s,a) slice gives an automaton accepting $F_{s,a}$. □

By making s and a parameters, the comparison of letters at two distant places has turned into a condition on a single column. For the same reason, $\mathrm{pred}(0) = s$ is also made a parameter.

Definition 6.7. Let $\mathcal{C}^0_{s,a}$ be the language obtained from the (s,a) slice by removing $F_{s,a}$. The superscript 0 means "with the firings whose tail is empty removed"; it is not the number of a level or of a rank. By Lemma 3.2, this is, with parameters $s, a, \bar{p}$, an M-regular language. Further, let SeedOnly($\bar{p}$) be the formula stating that the candidate contains a word consisting of the seed only (a word of length 1). It is obtained by applying Lemma 3.3 to the intersection of the candidate with D (the set of all words of length 1).

6.2.4 The case of rank 0

Theorem 6.8 (Avoidance problem at rank 0)**.** *One can build from syntax alone an M-formula $\Phi_0(\bar{p})$ such that the following holds: whenever the body of every candidate word has rank 0 under a valuation $\bar{p}$, the avoidance problem $(M, \mathcal{R}, \mathcal{C})$ has a solution if and only if*

$$M \models \Phi_0(\bar{p}), \qquad \Phi_0(\bar{p}) = \mathrm{SeedOnly}(\bar{p}) \ \vee \ \exists s\, \exists a.\ \mathrm{NonEmpty}_{\mathcal{C}^0_{s,a}}(s, a, \bar{p})$$

holds.

Proof. Suppose a firing with a nonempty tail occurs. By (iii), $w[t{:}]$ is a nontrivial border of $w[j{:}]$, which contradicts rank 0. Hence only a firing with an empty tail can occur.

A word s consisting of the seed only has an empty body, so there is no position satisfying $0 \le j < t \le 0$, and no row fires. Hence the candidate contains such a word if and only if SeedOnly holds, and if it does, there is a solution.

A word $s\,w$ with a nonempty body belongs to exactly one slice (s,a). For that word,

$$\text{no row fires} \iff \text{no row fires with an empty tail} \iff s\,w \notin F_{s,a} \iff s\,w \in \mathcal{C}^0_{s,a}$$

holds. The first equivalence was shown above, the second is Lemma 6.6, and the third holds by definition. By Lemma 3.3, $\mathcal{C}^0_{s,a}$ is nonempty if and only if $M \models \mathrm{NonEmpty}_{\mathcal{C}^0_{s,a}}(s, a, \bar{p})$. Putting these together gives the claim. □

In Φ_0, the symbols $\vee$ and $\exists s\, \exists a$ are ordinary logical symbols. The variables s and a remain free while the formula is being assembled, and are finally bound by quantifiers of M. Only at this quantification do we look at all the slices together. Also, the hypothesis of the theorem (the rank of the body is 0) is not something the procedure checks; where the theorem is applied, it is guaranteed by the rank lemma and strict descent (Section 6.1).

Example 6.9. At the bottom stage of Section 5.1, Example 5.7 ($M = \Sigma$, candidate $\sigma\,\mathrm{Dead}$, one cap row), the cap row requires a nonempty tail, so $F_{s,a}$ is empty. The only nonempty slices are those with $s = \sigma$: the $(\sigma, 1)$ slice is $\sigma\, 0^*\{11, 101, 111\}$, and the $(\sigma, 0)$ slice is $\sigma\, 0^*\{100, 110\}$. The rank of Dead is at most 2, so at this stage Theorem 6.8 cannot be used directly. In the next section we encode $\mathcal{C}^0_{s,a}$ to lower the rank.

6.3 Block encoding

In this section we move the firing with a nonempty tail of an avoidance problem to the firing of an avoidance problem of the same form over a changed alphabet. If the new body is nonempty, the rank of the body drops by at least 1.

From now on, fix a structure M (with domain D), parameters $\bar{p}$, a set of rows $\mathcal{R}$ and a candidate $\mathcal{C}$. Further fix slice parameters $s, a \in D$, and consider only the words of $\mathcal{C}^0_{s,a}$ (Section 6.2). These words $s\,w$ have a nonempty body w with last letter a, and no row fires on them with an empty tail.

6.3.1 Decomposition

Since the last letter of w is a, w decomposes uniquely as

$$w = U_1\, a\, U_2\, a \cdots U_m\, a \qquad (m \ge 1,\ \text{each } U_i \in D^* \text{ does not have } a \text{ as a letter})$$

We call U_i a block. A block may be empty.

Definition 6.10 (Block encoding)**.** Let $D' = D^*$ be the new alphabet (the empty block is also one letter of D'). The encoding of $s\,w$ is defined as the word over D'

$$\mathrm{enc}_{s,a}(s\,w) = U_1\,(U_2, \ldots, U_m)$$

The new seed is U_1, and the new body is $v = (U_2, \ldots, U_m)$.

Since s and a are parameters, the original word $s\,w$ can be recovered from the encoding. The first block is made the seed rather than put into the body so that each block of the new body has a delimiter a just before it (this is used in the proof of strict descent below). When $m = 1$, the new body is empty.

6.3.2 Strict descent of the rank

Lemma 6.11 (Strict descent)**.** *If $m \geq 2$, then* $\mathrm{rank}(v) + 1 \leq \mathrm{rank}(w)$*. Here* $\mathrm{rank}(v)$ *is the rank of v viewed as a word over D'.*

Proof. In v, take a nonempty suffix $(U_k, \ldots, U_m)$ $(2 \leq k \leq m)$, and let the lengths of its nontrivial borders be $\ell_1 > \cdots > \ell_r$. Since $k \geq 2$, just before U_k there is, in w, an occurrence of a. In w, consider the suffix $z = a\,U_k\,a \cdots U_m\,a$. A block border of length ℓ, $(U_k, \ldots, U_{k+\ell-1}) = (U_{m-\ell+1}, \ldots, U_m)$, lifts to the border of z

$$a\,U_{m-\ell+1}\,a \cdots U_m\,a \;=\; a\,U_k\,a \cdots U_{k+\ell-1}\,a$$

(the left-hand side is a suffix of z, and the right-hand side is a prefix of z). Further, the one-letter word a is also a border of z. These $r+1$ borders have pairwise different lengths, and all of them are nonempty and shorter than z. Hence z has at least $r+1$ nontrivial borders, and $\mathrm{rank}(w) \geq r+1$. Since the suffix $(U_k, \ldots, U_m)$ is arbitrary, $\mathrm{rank}(w) \geq \mathrm{rank}(v) + 1$. □

For example, if $a = 1$ and $w = 0\,1\,0\,1\,0\,1$, then $v = ([0], [0])$, and v has one nontrivial border $([0])$. This lifts, in the suffix of w written $z = 1\,0\,1\,0\,1$, to the border $1\,0\,1$, and 1 $(= a)$ is also a border of z, so z has two nontrivial borders.

6.3.3 Decomposition of the border condition

We assign the positions of w to blocks as follows. If the block U_i starts, in w, at position b_i and the a right after it is at position e_i, then the positions $b_i, \ldots, e_i$ belong to U_i. That is, placed just before the delimiter a, the position e_i also belongs to U_i. Each of the positions $0, \ldots, |w| - 1$ belongs to exactly one block.

Example 6.12 (Decomposition and correspondence of positions)**.** We first look at the content of the next lemma on an example. Let $D = \{0, 1\}$ and $a = 1$, and consider the body

$$w = \underbrace{0\,0\,0}_{U_1}\,1\,\underbrace{0}_{U_2}\,1\,\underbrace{0\,0}_{U_3}\,1\,\underbrace{0}_{U_4}\,1 \qquad \text{(positions } 0, \ldots, 10\text{)}$$

The ranges of the blocks are U_1: positions 0–3, U_2: 4–5, U_3: 6–8, U_4: 9–10 (including, for the delimiter 1, its position e_i). Take the pair of positions $(j, t) = (2, 7)$. Then j belongs to U_1, t belongs to U_3, and

$$U_1 = \underbrace{0\,0}_{\pi}\,\underbrace{0}_{S}, \qquad U_3 = \underbrace{0}_{\pi'}\,\underbrace{0}_{S}$$

Placing the two suffixes side by side, they agree up to the first 1 (the delimiter) as $S\,1 = 0\,1$, and after that the block sequences $(U_2) = (0)$ and $(U_4) = (0)$ agree.

$$\begin{aligned} w[j{:}] = w[2{:}] \quad & \underbrace{0\,1}_{S\,a}\,\underbrace{0\,1}_{U_2\,a}\,0\,0\,1\,0\,1 \\ w[t{:}] = w[7{:}] \quad & \underbrace{0\,1}_{S\,a}\,\underbrace{0\,1}_{U_4\,a} \end{aligned}$$

Hence $w[7{:}] \sqsubseteq w[2{:}]$. After encoding, the new seed is $[000]$ and the new body is $v = ([0], [00], [0])$. The original pair of positions $(2, 7)$ corresponds to the new pair of positions $(i - 1, l - 1) = (0, 2)$. Of the new markers, L' goes right after U_1 (the seed), and R' goes right after U_3:

$$[000]\ L'\ [0]\ [00]\ R'\ [0].$$

The new border condition is $v[2{:}] = ([0]) \sqsubseteq v[0{:}] = ([0], [00], [0])$, which is exactly (iii) of the next lemma. The two letters compared by the new guard are those just before L' and R', $[000] = U_1$ and $[00] = U_3$. The relation H of the next section reads these two blocks aligned at the right end:

	Column 1	Column 2	Column 3
$\mathrm{rev}(U_1)$	0	0	0
$\mathrm{rev}(U_3)$	0	0	#
Role	S	last letters of π, π' (the guard is checked here)	rest of π

The common suffix S lies in the same columns, so the two letters compared by the original guard (the last letters of π and π') also lie in the same column 2.

Lemma 6.13 (Fiber correspondence)**.** *Let $0 \leq j < t < |w|$, and suppose that j belongs to the block U_i and t belongs to the block U_l. Then $w[t{:}] \sqsubseteq w[j{:}]$ if and only if all of the following three conditions hold.*

1. $i < l$.

2. *There are words* π, π', S *such that* $U_i = \pi S$, $U_l = \pi' S$, $j = b_i + |\pi|$, $t = b_l + |\pi'|$.

3. $(U_{i+1}, \ldots, U_{i+m-l}) = (U_{l+1}, \ldots, U_m)$.

Proof. ($\Rightarrow$) Both $w[j{:}]$ and $w[t{:}]$ end in a, so they contain a. Since $w[t{:}]$ is a prefix of $w[j{:}]$, the first occurrences of a in the two are at the same position (distance from the start). In $w[j{:}]$ the first a is at position e_i, and in $w[t{:}]$ the first a is at position e_l. Hence, setting $S = w[j{:}e_i] = w[t{:}e_l]$, both U_i and U_l end in S, and (ii) holds. If $i = l$, then $j = e_i - |S| = t$, contradicting $j < t$; hence from $t > j$ we get $l > i$ ((i)). After the first a, $w[e_l + 1{:}] = U_{l+1}a \cdots U_m a$ is a prefix of $w[e_i + 1{:}] = U_{i+1}a \cdots U_m a$. Between sequences of blocks ending in a, the prefix relation is equivalent to the prefix relation between the block sequences, so (iii) holds.

($\Leftarrow$) By (ii), $w[j{:}] = S\,a\,U_{i+1}\,a \cdots U_m\,a$ and $w[t{:}] = S\,a\,U_{l+1}\,a \cdots U_m\,a$. By (iii), the latter is a prefix of the former. □

Condition (ii) concerns only the two blocks U_i, U_l. Condition (iii) is a condition on the new body v, and can be written as $v[l-1{:}] \sqsubseteq v[i-1{:}]$ (in v, position k holds the letter U_{k+2}). This is the border condition of the new pair of positions $(i-1, l-1)$, and the letters just before these positions are exactly U_i and U_l. When $l = m$, the new tail is empty.

6.3.4 The new guard and the new rows

Fix a row $(T, G) \in \mathcal{R}$, and let the set of states of T be Q. The predicates below are built for each row. When the predicates of different rows are collected into one structure, each predicate symbol carries the identifier of its original row, but we omit this in the notation.

Overall division of labor. When the original row fires at positions (j, t), the word read by T splits into the following five intervals, at the boundaries of the blocks containing j and t, U_i, U_l.

Interval	Part read by T	What checks it at the new level
1	seed s and $U_1 a \cdots U_{i-1} a$	T' (the seed by Start, the blocks by Run; if $i = 1$, only the seed)
2	inside U_i: $\pi\,L\,S\,a$	H
3	$U_{i+1} a \cdots U_{l-1} a$	T'
4	inside U_l: $\pi'\,R\,S\,a$	H
5	$U_{l+1} a \cdots U_m a$	T'

Let the states of T at the boundaries of the intervals be, in order, r_0, r_1, t_0, t_1 (the boundaries between intervals 1 and 2, 2 and 3, 3 and 4, and 4 and 5). A new row has these four states and a bit ι (whether U_i is the seed) as indices. T' reads ordinary blocks by Run, consumes the left target block U_i as one letter without looking at its contents (jumping from state r_0 to r_1), reads the intermediate blocks, treats the right target block U_l in the same way (from t_0 to t_1), and reads the rest. H checks the contents of U_i, U_l (intervals 2 and 4), the common suffix S, and the original guard. The two share the same four index states, and this allows the runs of the five intervals to be glued into a single run. If only H or only T' holds, this does not give the original firing (see Example 6.20).

Definition 6.14 (Predicates on blocks)**.** Let $q, r \in Q$ and $U, V \in D' = D^*$.

- $\mathrm{Run}_{q,r}(U)$: T, from state q, reads $U\,a$ (which contains no marker) and arrives at state r.
- $\mathrm{NoA}(U)$: U does not contain a.
- Start_q: T, from some initial state, reads the seed s and arrives at state q (a 0-ary relation symbol for each q; a parameter-only condition).
- Final_q: q is an accepting state of T (a 0-ary relation symbol for each q).
- For $(r_0, r_1, t_0, t_1) \in Q^4$ and $\iota \in \{0, 1\}$, $H^{(r_0,r_1,t_0,t_1,\iota)}(U, V)$: there are words π, π', S such that $U = \pi S$, $V = \pi' S$, T, from r_0, reads $\pi\,L\,S\,a$ and arrives at r_1, T, from t_0, reads $\pi'\,R\,S\,a$ and arrives at t_1, and $G(x_L, x_R; \bar{p})$ holds. Here x_L is the last letter of π (if π is empty, it is, when $\iota = 1$, s, and when $\iota = 0$, a), and x_R is the last letter of π' (if π' is empty, a).

The letters x_L and x_R correspond to $\mathrm{pred}(j)$ and $\mathrm{pred}(t)$ in the original word. When π is empty, $j = b_i$, and the letter just before it is, if $i \geq 2$, the delimiter a, and if $i = 1$, the seed s. When π' is empty, $l \geq 2$, so the letter just before it is a.

Lemma 6.15. *Each of the above predicates is given, with parameters* $s, a, \bar{p}$*, by a column-local synchronous automaton over* M *(Section 3.5).*

Proof. $\mathrm{Run}_{q,r}$ and NoA have one tape: the former follows the transitions of T in reverse, and the latter checks $x \neq a$ at each column. Start_q and Final_q have zero tapes (they are parameter-only formulas). H is given by an automaton that places U and V on two tapes aligned at the right end and reads them from the right as follows. First, from the pair of states (r_1, t_1), it follows, in T, the a transition one step backward on both sides (since a is a parameter, no column is read). Next, in the columns of the common suffix S, it follows the transitions of T backward on both sides while checking that the letters on the two tapes are equal. It guesses the end of S nondeterministically and follows backward the L transition on the left and the R transition on the right. The next column holds the last letters of π and π', so in that column it checks $G(x_1, x_2; \bar{p})$ (an empty side is padding, and s or a is used instead). After that it follows π and π' backward separately, and checks that the ends of the tapes are reached at r_0 and t_0, respectively. When $\pi = \pi' = \varepsilon$, no column remains after S (this includes the case $U = V = \varepsilon$). In this case, at the end of the input, it imposes as an acceptance condition (a parameter-only formula) that the states reached by following the L and R transitions backward are r_0, t_0, respectively, and also $G(s, a; \bar{p})$ $(\iota = 1)$ or $G(a, a; \bar{p})$ $(\iota = 0)$. Marker transitions read no input, so their conditions are absorbed into adjacent edges or the acceptance condition. The moves of this automaton fall into three kinds. On an edge that reads letters, it checks the original transition of T and, where needed, the guard with the empty side replaced by s or a (if only one side is empty, the guard also depends on the letter in the current column). The transition reading the final a in T and the guard in the case where both sides are empty are parameter-only conditions. L and R are treated as simulated transitions that consume no input. (The transition reading the seed s in T is handled not by H but by Start_q.) Every edge relates only letters in the same column and parameters. □

Definition 6.16 (Outside controller)**.** For $(r_0, r_1, t_0, t_1, \iota)$, over $D' \cup \{L, R\}$ we define the symbolic automaton $T'^{(r_0,r_1,t_0,t_1,\iota)}$ as follows. Its states are before(b, q) $(b \in \{0, 1\},\ q \in Q)$, waitLeft, middle$(q)$, waitRight, after$(q)$.

- The initial states are the states before$(1, q)$ for which Start_q holds. Here $b = 1$ means that no block has been read yet.
- From before(b, q), go with label $\mathrm{Run}_{q,r}(U)$ to before$(0, r)$.
- When $q = r_0$ and $b = \iota$, from before(b, q), go with label $\top$ (skipping a block) to waitLeft.
- From waitLeft, go with marker L to middle(r_1).
- From middle(q), go with label $\mathrm{Run}_{q,r}(U)$ to middle(r).
- From middle(t_0), go with label $\top$ to waitRight.
- From waitRight, go with marker R to after(t_1).
- From after(q), go with label $\mathrm{Run}_{q,r}(U)$ to after(r).
- The accepting states are the states after(q) for which Final_q holds.

The new row is $\big(T'^{(r_0,r_1,t_0,t_1,\iota)},\ H^{(r_0,r_1,t_0,t_1,\iota)}\big)$. The set of these rows, over all rows of $\mathcal{R}$ and all $(r_0, r_1, t_0, t_1, \iota)$, is denoted by $\mathcal{R}'$.

$\mathcal{R}'$ is a finite set with $2|Q|^4$ rows for each original row. T' does not look at the contents of the two skipped blocks. Those contents are checked by the guard H. The two are connected only by the four index states (r_0, r_1, t_0, t_1) and the bit ι.

Lemma 6.17 (Correspondence of firing)**.** *Let $s\,w \in \mathcal{C}^0_{s,a}$. Some row of $\mathcal{R}$ has a firing with a nonempty tail on $s\,w$ if and only if some row of $\mathcal{R}'$ fires on $\mathrm{enc}_{s,a}(s\,w)$. Moreover, the positions (j, t) of the former and the positions of the latter correspond as follows: with j, t contained in the blocks U_i, U_l, the latter positions are $(i - 1, l - 1)$.*

Proof. $(\Rightarrow)$ Suppose the row (T, G) fires on $s\,w$ at positions (j, t) $(t < |w|)$, and let $j \in U_i$, $t \in U_l$. By Lemma 6.13, (i)–(iii) hold. Take a path on which T accepts $s\,w[:j]\,L\,w[j{:}t]\,R\,w[t{:}]$. This word is divided as

$$s \cdot U_1 a \cdots U_{i-1} a \cdot \pi\, L\, S\, a \cdot U_{i+1} a \cdots U_{l-1} a \cdot \pi'\, R\, S\, a \cdot U_{l+1} a \cdots U_m a$$

At the four division points (just before π, just after $S\,a$, just before π', just after $S\,a$), call the states of the path r_0, r_1, t_0, t_1, and let $\iota = [i = 1]$. Then $H^{(r_0,r_1,t_0,t_1,\iota)}(U_i, U_l)$ holds (its guard condition is exactly the original guard (ii)). Moreover, $T'^{(r_0,r_1,t_0,t_1,\iota)}$ accepts $U_1 \cdots U_i\, L\, U_{i+1} \cdots U_l\, R\, U_{l+1} \cdots U_m$, by following each interval of the above path along Run edges. The new pair of positions is $(i - 1, l - 1)$, and $i < l$. The letters just before these positions are U_i and U_l (when $i = 1$, the new seed U_1). The new border condition is (iii) of Lemma 6.13. Hence the new row fires at positions $(i - 1, l - 1)$.

$(\Leftarrow)$ Suppose the new row (T', H) fires at positions $(i - 1, l - 1)$. An accepting path of T' passes through the edges skipping U_i and U_l exactly once each, and reads the other blocks along Run edges. From $H(U_i, U_l)$

we get π, π', S and two subpaths of T. Gluing these with the interval paths given by the path of T' yields a path on which T accepts $s\, w[:j]\, L\, w[j{:}t]\, R\, w[t:]$. Here $j = b_i + |\pi|$ and $t = b_l + |\pi'|$. The guard (ii) is exactly the condition inside H on G. From the new border condition and, in H, $U_i = \pi S$, $U_l = \pi' S$, Lemma 6.13 gives the original border condition. Since $t \le e_l < |w|$, this is a firing with a nonempty tail. □

6.3.5 The new avoidance problem

Definition 6.18. The new structure $M' = M'[s, a, \bar{p}]$ is the structure with domain $D' = D^*$ and the predicates $\mathrm{Run}_{q,r}$, $H^{(r_0,r_1,t_0,t_1,\iota)}$, NoA, Start_q, Final_q (for all rows). The new candidate $\mathcal{C}'$ is the set of encodings of the words of $\mathcal{C}^0_{s,a}$. To accept $\mathcal{C}'$, take $\mathcal{C}^0_{s,a}$ and fix a deterministic symbolic automaton $\mathcal{A}$ accepting it (Lemma 3.2), and, for its states q, r, add the following predicates to M' as well: "$\mathcal{A}$, from q, reads $U\, a$ and arrives at r" (one tape), "$\mathcal{A}$, from the initial state, reads the seed s and arrives at q" and "q is an accepting state of $\mathcal{A}$" (parameter-only conditions). $\mathcal{C}'$ is accepted, using these predicates and NoA, by a symbolic automaton, imitating $\mathcal{A}$ block by block, over M'. This automaton reads at least one block, counting the seed block U_1, and imposes $\mathrm{NoA}(U)$ on the label of each edge reading a block. The original seed s is handled by the start condition (the state of $\mathcal{A}$ after reading s), and the new seed U_1 is handled as the first block read, together with the effect of the delimiter a.

For the same reason as in Lemma 6.15, M' is, with parameters $s, a, \bar{p}$, an automatic structure over M. The labels of T' and the labels of the automaton accepting $\mathcal{C}'$ are formulas of M'. Hence $(M', \mathcal{R}', \mathcal{C}')$ is an avoidance problem of the same form as in Section 6.2.

For the fact that $\mathrm{Run}_{q,r}$ and H are predicates of M' whose meaning is given by automata over M, see "Names and contents of predicates" in Section 6.1.

Theorem 6.19 (One step of block encoding)**.** *For all s, a, $\mathcal{C}^0_{s,a}$ contains a word on which no row of $\mathcal{R}$ fires if and only if the avoidance problem $(M'[s, a, \bar{p}], \mathcal{R}', \mathcal{C}')$ has a solution. Moreover, if the bodies of $\mathcal{C}$ all have rank at most $h \ge 1$, then the bodies of $\mathcal{C}'$ all have rank at most $h - 1$.*

Proof. No row fires with an empty tail on a word of $\mathcal{C}^0_{s,a}$, so no row fires on it if and only if no row fires on it with a nonempty tail. By Lemma 6.17, this is equivalent to no new row firing on its encoding. The encoding is a bijection from $\mathcal{C}^0_{s,a}$ to $\mathcal{C}'$, so the first part holds. The second part follows from Lemma 6.11 if $m \ge 2$, and from the fact that the new body is empty if $m = 1$. □

Example 6.20. In Section 5.1, Example 5.7, name the states of the cap row T as follows: α reads the anchor, $\beta_0, \beta_1, \beta_2$ ($\beta_0 \xrightarrow{1} \beta_1 \xrightarrow{0} \beta_2$) read the root, and γ, δ read the tail (γ means that the tail is still empty, δ accepts after one or more letters). The state after reading the seed σ is α, and $\alpha \xrightarrow{L} \beta_0$, $\beta_2 \xrightarrow{R} \gamma$. We work in the slice $(\sigma, 1)$.

The decomposition of $w = 0^n 101$ is $U_1 = 0^n$, $U_2 = 0$, and the encoding is $[0^n] \mid [0]$. The original cut $(n, n+2)$ lies at the end of U_1 ($\pi = 0^n$, $S = \varepsilon$) and at the end of U_2 ($\pi' = 0$, $S = \varepsilon$). T, from α, reads $0^n\, L\, 1$ and arrives at β_1, and, from β_1, reads $0\, R\, 1$ and arrives at δ. Hence $H^{(\alpha,\beta_1,\beta_1,\delta,1)}([0^n], [0])$ holds, and the new row fires at positions $(0, 1)$. Since the new body has length 1, this is a firing with an empty tail, and it is removed by the subtraction at the next stage.

The decomposition of $w = 0^n 11$ is $U_1 = 0^n$, $U_2 = \varepsilon$, and the encoding is $[0^n] \mid [\,]$. The only possible new pair of positions is $(0, 1)$. The T' of a new row firing at this position skips the first block, so $r_0 = \alpha$, $\iota = 1$, and there is no block in between, so $t_0 = r_1$. On the other hand, in $H([0^n], [\,])$ we have $V = \varepsilon$, so $\pi' = S = \varepsilon$, $\pi = 0^n$, and T, from α, reads $0^n\, L\, 1$ and arrives at $r_1 = \beta_1$. Hence we would need $t_0 = \beta_1$, but β_1 has no R transition (the root 10 stops at 1). Therefore no new row, with any indices, fires on $[0^n] \mid [\,]$. (Without restricting the indices, for example $H^{(\alpha,\beta_1,\beta_2,\delta,1)}([0^n], [\,])$ holds. But with these indices $t_0 = \beta_2 \ne r_1$, so T' does not accept. This example shows that H alone is not enough, and that the check must be made together with the T' of the same indices.)

6.4 Lowering and assembling the tower

By Section 6.3 (Theorem 6.19), the avoidance problem at level k moves, for each slice (s, a), to an avoidance problem on the level $k+1$ structure $M'[s, a, \bar{p}]$. In this section we give a method for translating a sentence built at level $k+1$ (a sentence of M') into a formula of level k, and use it to assemble the whole procedure.

6.4.1 Lowering lemma

Lemma 6.21 (Lowering)**.** *Let N be, with parameters $\bar{p}$, an automatic structure over M whose signature consists only of finitely many relation symbols (including 0-ary ones) (Section 3.5). Over the signature of N, from any*

first-order sentence Φ *one can build from syntax alone a formula with free variables among* $\bar{p}$*, namely the* M*-formula* $\mathrm{Down}(\Phi)(\bar{p})$*, such that for all* $\bar{p}$

$$N[\bar{p}] \models \Phi \iff M \models \mathrm{Down}(\Phi)(\bar{p})$$

holds.

This is the result of Bès [2] that "the first-order theory of an automatic structure over M reduces to the theory of M", put in a form uniform in the parameters. Uniformity is needed because $\bar{p}$ remains a free variable in the formula and is bound later by a quantifier of the lower level.

Proof sketch. Within Φ, take each subformula $\psi(u_1, \ldots, u_n)$ (where u_k is a variable denoting an element of D^*). To accept the convolutions of the tuples satisfying ψ, we build an n-tape column-local synchronous automaton $\mathcal{A}_\psi$ by induction on the construction of ψ. We also allow the automata to have edges that read no input and have parameter-only conditions. Such edges can be removed by a Boolean combination over finitely many paths. The convention that no data appears after padding on any tape is enforced by finitely many flags kept in the states.

- *Atomic formulas*: use the given automaton and skip the unused tapes. If unused tapes continue after all the selected tapes have ended, move, under the acceptance condition of that automaton, to a waiting phase, and skip the rest of the unused tapes while checking that the selected tapes remain padding. An atomic formula in which the same variable appears at several argument positions (for example $R(u, u)$) is handled by replacing the duplicated arguments by new variables, connecting them by word equality, and then quantifying existentially. Word equality is represented by an automaton that reads the same letter on two tapes and ends on both at the same time.
- $\wedge$, $\vee$: product and union. The two components share the same parameter values.
- $\neg$: after the determinization by subset construction and cells as in Lemma 3.2, negate the acceptance condition. Unsatisfiable cells are also kept, without being decided. Finally, take the intersection with the convolutions satisfying the padding convention.
- $\exists u_k$: remove the tape of u_k (projection). In the label of each edge, only when u_k is not padding in that column, close just the letter of u_k with $\exists$. If all other tapes are padding in that column, that edge becomes an edge that reads no input. After the projection, the states still keep the padding flags of all the original tapes, including the removed one. This forbids a visible tape from resuming after it has ended, while allowing a part in which only hidden tapes continue. Also, an accepting path after projection can be turned back into an original accepting path by supplying the letters that justify the existential quantification on each edge. For example, take the 2-tape automaton representing "$|U| \leq |V|$" (with a state p reading columns that have letters on both tapes, and, reading after the end of U the columns $(\#, y)$, a state p'). When the tape of V is removed, the edges of p become edges reading a letter of U (with condition $\exists y.\, \top$), and the edges of p' become edges that read no input. The result is an automaton accepting every U. Concretely, for $U = 0$, $V = 01$, the convolution of the reversed words is $(0, 1)(\#, 0)$. When V is removed, the first column becomes an edge reading, from U, the letter 0, the second column becomes an edge that reads no input, and the padding flag remembers that "U ended at the first column".

Since Φ is a sentence, we finally obtain a 0-tape automaton. Its only input is the empty convolution, and whether it accepts this input expresses the truth value of Φ. The formula expressing its nonemptiness, in the same form as in Lemma 3.3, as a formula of M, is denoted by $\mathrm{Down}(\Phi)(\bar{p})$.

The key to the correctness of each step is column-locality. The label of each edge relates only letters in the same column and parameters. Hence, if the labels of the edges on a path can each be satisfied, we can choose letters independently for each column and actually construct a tuple of words accepted along that path. The parameters $\bar{p}$ have a common value along the whole path, and are never quantified edge by edge. The details for the case without parameters are in [2], Proposition 2.6–2.7 and Theorem 3.9. The parameter-uniform version is formalized in Lean (modules `SymbolicAutomata`, `ConvolutionShapes`, `SymbolicProjection`, `SymbolicPreimage`; Section 8). □

In Section 6.3, the structure $M'[s, a, \bar{p}]$ is, by Lemma 6.15, with parameters $(s, a, \bar{p})$, an automatic structure over M. Hence a sentence of M' can be translated by lowering, with $s, a, \bar{p}$ as free variables, into a formula of M.

Example 6.22 (A small example of lowering)**.** Let $M = \Sigma = \{0, 1\}$, and let T be a 2-state NFA ($q \xrightarrow{0} q$, $q \xrightarrow{1} r$). Take the delimiter letter a as a parameter, and translate the sentence of M'

$$\Psi \;=\; \exists U.\, \big(\mathrm{NoA}(U) \wedge \mathrm{Run}_{q,r}(U)\big)$$

into a formula of M.

1. *Atomic formulas*: NoA is an automaton with one state (initial and acceptance conditions $\top$) whose loop has the label $x \neq a$. $\mathrm{Run}_{q,r}$ is, in order to read $\mathrm{rev}(U)$, the automaton with the edges of T reversed. Absorbing the reversed transition that reads the final a into the initial condition gives $\mathrm{Init}_q = (a = 1)$, $\mathrm{Init}_r = \bot$, $\mathrm{Fin}_q = \top$, $\mathrm{Fin}_r = \bot$, and the loop of q has the label $x = 0$. Removing the unreachable state r leaves one state, q.

2. $\wedge$: taking the product gives one state, initial condition $a = 1$, acceptance condition $\top$, and loop label $x \neq a \wedge x = 0$.

3. $\exists U$: removing the tape of U turns the loop into an edge that reads no input, and its condition becomes $\exists x.\, (x \neq a \wedge x = 0)$ (a is not quantified).

4. *Nonemptiness*: the only simple path is the path of length 0, so $\mathrm{Down}(\Psi)(a) = (a = 1)$.

The resulting formula contains neither Run nor NoA; only conditions on letters of Σ and on parameters remain. The meaning also agrees: T, from q, reads $U\,a$ and arrives at r exactly when $U \in 0^*$ and $a = 1$, and $U = \varepsilon$ always satisfies NoA. In general, the outer shape of the translated formula is a disjunction, over the simple paths, of the conjunction of "the initial condition, the acceptance condition, and the condition that each edge on the path is usable". Since projection existentially quantifies the letters of the removed tape, the quantification over blocks $\exists U$ is replaced by quantification over the letters of each column. However, general formulas of M (possibly containing negation and quantifiers) remain inside each condition.

Iterating satisfiability and nonemptiness. We restate what lowering does, including the case of stacked levels. In M', take a formula $\psi(u_1, \ldots, u_n)$. The statement "in M', ψ is satisfied by some $(u_1, \ldots, u_n)$" (satisfiability in the fixed structure M') is equivalent to the nonemptiness of the automaton assembled along the construction of ψ over M, namely $\mathcal{A}_\psi$, that is, to the truth of $\exists u_1 \cdots \exists u_n.\ \psi$. By the multi-tape version of Lemma 3.3 (the condition of each edge is taken to be $\exists \bar{x}.\, \theta$, and the padding convention is taken to be already enforced by flags in the states), that nonemptiness is expressed by a formula of M. For a sentence (a formula without free variables) Φ, what remains at the end is an automaton with 0 tapes, whose only input is "reading nothing", so nonemptiness is the same as Φ being true. In a formula, $\neg$ and $\forall$ ($= \neg\exists\neg$) are handled by complement automata, so even for a sentence containing $\neg$ or $\forall$, what is asked at the end is the nonemptiness of a single automaton with 0 tapes. A $\neg$ attached to a subformula without free variables (in particular to a whole sentence) may be treated as follows. An automaton with 0 tapes, after removal of the edges that read no input, just represents one truth value, so the nonemptiness of its complement is equivalent to the negation of the original nonemptiness formula. We therefore adopt below the convention $\mathrm{Down}(\neg\Phi) := \neg\mathrm{Down}(\Phi)$ (this is an equivalent formula, different from the one produced by the construction of Lemma 6.21). The nonemptiness formula obtained in this way is a formula of M, and the predicates of M (given by automata one level below) may appear in it as names. So we perform the same translation once more. However, the input of the next lowering must be a sentence of the structure one level below. Among the parameters remaining as free variables in the translated formula, the s, a introduced at that level are quantified by $\exists$ to form a sentence (the recursion of Theorem 6.24), and the parameters inherited from lower levels are regarded as part of the interpretation of that structure. With this sentence formation inserted at each stage, the outline is the following chain:

$$M_h\text{: truth of a sentence} \iff M_{h-1}\text{: nonemptiness of an automaton}$$
$$\iff M_{h-1}\text{: truth of a formula} \iff \cdots \iff \Sigma\text{: truth of a formula}$$

(for fixed parameter values; the translated formula has only the parameters as free variables). In the recursion of Theorem 6.24, $\exists s\, \exists a$ and SeedOnly are additionally inserted at each level, so the above chain shows only the skeleton. One translation removes all predicate names of the top level. Predicate names of the level one below may remain in the formula through the labels of the automata, but they too disappear at the next translation.

Example 6.23 (Two stages of lowering)**.** Let $M_0 = \Sigma = \{0, 1\}$, and let $a \in \Sigma$ be a parameter. Every language below is determined by "each letter (block) satisfies some condition" or by its negation, so it is closed under reversal. Hence reading a word from the right is the same as reading it from the left.

- At level 1 (domain Σ^*), the predicate $P(U)$: "U is nonempty and does not contain a". Its meaning is given, over M_0, by the automaton $\mathcal{A}_P$: states n_0, n_1, edges $n_0 \xrightarrow{x \neq a} n_1$ and $n_1 \xrightarrow{x \neq a} n_1$, initial state n_0, accepting state n_1.

- At level 2 (domain $(\Sigma^*)^*$), the predicate $Q(V)$: "V consists of at least one block, and each block satisfies P". Its meaning is given, over M_1, by the automaton $\mathcal{A}_Q$: states s_0, s_1, edges $s_0 \xrightarrow{P(y)} s_1$ and $s_1 \xrightarrow{P(y)} s_1$, initial state s_0, accepting state s_1. Since $\mathcal{A}_Q$ is an automaton over M_1, its label $P(y)$ is an atomic formula of M_1 (y is a letter of level 1, that is, a block).

We translate the sentence $\Phi = \exists V.\, Q(V)$ (a sentence of M_2).

First stage ($M_2 \to M_1$): replace the atomic formula $Q(V)$ by $\mathcal{A}_Q$ and project out the tape of V; the edge condition becomes $\exists y.\, P(y)$. The only simple path is $s_0 \to s_1$, so $\mathrm{Down}_1(\Phi) = \exists y.\, P(y)$. This is a formula of M_1, and the name of the predicate P remains.

Second stage ($M_1 \to M_0$): replace the atomic formula $P(y)$ by $\mathcal{A}_P$ and project out the tape of y; the edge condition becomes $\exists x.\, x \neq a$. The only simple path is $n_0 \to n_1$, so $\mathrm{Down}_0(\mathrm{Down}_1(\Phi))(a) = \exists x.\, x \neq a$. This is a formula of Σ, and no predicate name remains. Over $\Sigma = \{0, 1\}$ it is true for every a, which agrees with the meaning "there is a nonempty sequence of nonempty blocks not containing a" of Φ.

The case with $\neg$: let $P'(U)$ be "U does not contain a" (it may be empty; one state n, loop $x \neq a$, initial and accepting), let $Q'(V)$ be "every block of V satisfies P'" (one state s, loop $P'(y)$, initial and accepting), and translate $\Phi' = \forall V.\, Q'(V) = \neg\exists V.\, \neg Q'(V)$. $\neg Q'(V)$ is represented as follows: determinize $\mathcal{A}_{Q'}$ as in Lemma 3.2, obtaining $\{s\} \xrightarrow{P'(y)} \{s\}$, $\{s\} \xrightarrow{\neg P'(y)} \emptyset$, $\emptyset \xrightarrow{\top} \emptyset$ ($\emptyset$ is a sink state), and negate the acceptance condition (the accepting state is $\emptyset$). By projection and nonemptiness, $\exists V.\, \neg Q'(V)$ is translated into $\exists y.\, \neg P'(y)$, and using the above convention for the outer $\neg$ gives $\mathrm{Down}_1(\Phi') = \neg\, \exists y.\, \neg P'(y)$. Similarly, translating $\exists y.\, \neg P'(y)$ gives $\exists x.\, x = a$, and we obtain $\mathrm{Down}_0(\mathrm{Down}_1(\Phi'))(a) = \neg\, \exists x.\, x = a$. Since $a \in \Sigma$, this is false. This agrees with the meaning of Φ': blocks containing a (for example the one-letter word a) form a sequence V that does not satisfy Q', so "every sequence satisfies Q'" is false.

The guard in actual formulas. In the formulas arising from the avoidance problem, the guard appears as follows. At level $k+1$, the automaton $F_{s',a'}$ representing firing with an empty tail (Lemma 6.6) has on its edge labels, with the right argument of the guard fixed to the slice parameter a', the formula $G'(x, a')$. At level $k+1$ the guard is $G' = H$, so this label is $H(x, a')$. Here both x and a' are letters of level $k+1$, that is, words over D. When a sentence of level $k+1$ is translated by lowering into a formula of M, $H(x, a')$ is replaced by a 2-tape automaton with x and a' as its tapes (Lemma 6.15). Since a' is bound, at the end of building the formula of level $k+1$, by $\exists a'$, it disappears by projection during the translation.

6.4.2 The tower

Theorem 6.24 (Uniform rank theorem)**.** *Let $h \geq 0$. From a structure M, a finite set of rows $\mathcal{R}$ and a candidate $\mathcal{C}$ (with parameters $\bar{p}$), one can build from syntax alone an M-formula $\Phi_h(\bar{p})$ such that the following holds: whenever the body of every candidate word has rank at most h under a valuation $\bar{p}$, the avoidance problem $(M, \mathcal{R}, \mathcal{C})$ has a solution if and only if $M \models \Phi_h(\bar{p})$.*

Proof. By induction on h. For $h = 0$, take the Φ_0 of Theorem 6.8.

Let $h \geq 1$. With the slice parameters s, a as variables, build, as in Section 6.3, the avoidance problem $(M'[s, a, \bar{p}], \mathcal{R}', \mathcal{C}')$. Applying the induction hypothesis $(h-1)$ to it gives the M'-sentence Φ'_{h-1}. Here $s, a, \bar{p}$ are part of the interpretation of M', and are not free variables of Φ'_{h-1}. Applying lowering, we set

$$\Phi_h(\bar{p}) \;=\; \mathrm{SeedOnly}(\bar{p}) \;\vee\; \exists s\, \exists a.\, \mathrm{Down}(\Phi'_{h-1})(s, a, \bar{p})$$

We show correctness. Let $\bar{p}$ be a valuation under which the bodies of the candidate all have rank at most h. A solution of the avoidance problem is either a word consisting of the seed only or a word belonging to some slice (s, a). The existence of the former is equivalent to $\mathrm{SeedOnly}(\bar{p})$ (as in the proof of Theorem 6.8). For the latter, a word of the (s, a) slice is a solution if and only if it belongs to $\mathcal{C}^0_{s,a}$ and no row fires on it with a nonempty tail. In this case, the bodies of $\mathcal{C}'$ have rank at most $h-1$ (Theorem 6.19), and the following equivalences hold.

the (s, a) slice contains a solution	
$\iff$ $(M'[s, a, \bar{p}], \mathcal{R}', \mathcal{C}')$ has a solution	(Theorem 6.19)
$\iff$ $M'[s, a, \bar{p}] \models \Phi'_{h-1}$	(induction hypothesis)
$\iff$ $M \models \mathrm{Down}(\Phi'_{h-1})(s, a, \bar{p})$	(Lemma 6.21)

Existentially quantifying over s, a gives the claim. □

This construction only assembles formulas; it does not evaluate the truth of a formula at any intermediate level. The recursion depth is h, and the number of rows and the size of the formulas at each level are finite.

6.4.3 The whole procedure

For a restricted cap set expression E, consider the following procedure AvoidExists(E).

1. If $\Sigma = \emptyset$, then, since $\Sigma^* = \{\varepsilon\}$, return the truth value of "$\varepsilon \notin E$" and stop.

2. Compute Cand, Mid, the DFA of Cand, n_{acc}, n_{root}, Live, ShortCov (Sections 5.1 and 5.2).

3. If Live $\not\subseteq$ ShortCov, return true.

4. Let Dead = Cand $\setminus$ (Live $\cup$ Mid) and $h = \max(0, n_{\mathrm{acc}} - 2)$.

5. Choose a letter $\sigma \in \Sigma$, and apply Theorem 6.24 to $M = \Sigma$ (no parameters), the set of cap rows and the candidate σ Dead to build the sentence Φ_h. Since Σ is finite, the truth value of Φ_h can be evaluated by expanding the quantifiers over the finitely many letters of Σ. Return that truth value.

Theorem 6.25 (Decidability of universality of restricted cap set expressions)**.** AvoidExists(E) *is true if and only if there exists a word that does not belong to* E*. Hence it is decidable whether a restricted cap set expression is universal.*

Proof. The case $\Sigma = \emptyset$ is clear, so let $\Sigma \neq \emptyset$. If step 3 returns true, then by Corollary 5.15 (1), E is not universal. Otherwise, by Corollary 5.15 (2), E is not universal if and only if Dead contains a word without a cut (an avoider). By Lemma 6.4, this is equivalent to the avoidance problem $(\Sigma, \text{cap row}, \sigma\,\text{Dead})$ having a solution. The bodies of the candidate are exactly the words of Dead. By the rank lemma (Lemma 5.21), the words of Dead have rank at most h. Hence, by Theorem 6.24, this is equivalent to $\Sigma \models \Phi_h$. □

Corollary 6.26 (Computable bound on short avoiders)**.** *From* E *one can compute a number* sabBound(E) *such that, if a word not belonging to* E *exists, then such a word of length at most* sabBound(E) *exists.*

Proof. If AvoidExists(E) is false, let sabBound(E) = 0. If it is true, examine words in order of length (lexicographically within the same length), and let the length of the first word not belonging to E be sabBound(E). By Theorem 6.25 such a word exists, so this search always terminates. Whether each word belongs to E can be decided by a finite computation, by Section 5.1, Lemma 5.5 and so on. □

This decision procedure does not use the assumption "if an avoider exists, then one of length at most a computable bound exists". The bound is constructed a posteriori from the decision procedure.

6.4.4 Main theorem

Combining the reduction of Section 4 with Theorem 6.25, we obtain the following.

Theorem 6.27 (Main theorem)**.** *With one constructor (an arity* $d \geq 1$ *covariant* f*), non-structural subtype entailment is decidable for each of finite trees, regular trees and arbitrary trees.*

Proof. By Section 4, it suffices to decide satisfiability and, if satisfiable, to decide whether the two restricted cap set expressions E_ℓ, E_r are universal. The latter is decidable by Theorem 6.25. □

7 Examples

In this section, for the restricted cap set expression of Section 5.1, Example 5.7,

$$\Sigma = \{0, 1\}, \quad A = 0^*, \quad B = \{10\}, \quad C = 0^*1\Sigma^3$$

we follow, level by level, what the procedure AvoidExists (Section 6.4) computes. For the decision on Dead, the procedure does not enumerate candidate words. It assembles a formula and evaluates its truth value at the end. What follows rereads, on words, what the equivalence at each stage guarantees. In this section we put $T = T_0$, and for a new row, T' $(= T_1)$, H, Run follow the notation of Section 6.3. To avoid confusing the types of the objects, it helps to read with reference to the following table.

Symbol	What it is	Set it belongs to	Length at that stage
ε	Empty word of level 0	Σ^*	0
[ ]	Empty block (one letter of level 1)	$D_1 = \Sigma^*$	1 (as a word of level 1)
( )	Empty sequence of letters of level 1 (one letter of level 2)	$D_2 = D_1^*$	1 (as a word of level 2)
Empty body	A body containing no letter		0

Letters are written following the convention of Section 6.1: letters of level 1 (blocks) are written in square brackets, as in $[0^n]$, [0], [ ] (the empty block), and letters of level 2 (sequences of blocks) are written in parentheses, as in ( ) (the empty sequence). In the last subsection we also show the skeleton of the formula that is actually assembled.

7.1 Live and Dead

As seen in Section 5.2 and Section 5.3, $n_{\mathrm{acc}} = 4$, $\mathrm{Live} = 0^* \subseteq \mathrm{ShortCov}$, and

$$\mathrm{Dead} = 0^*\{11,\ 100,\ 101,\ 110,\ 111\}, \qquad h = 2$$

hold. The avoidance problem at level 0 has $M = \Sigma$, one cap row $(T, \top)$, and the candidates $\sigma\,\mathrm{Dead}$. The states of T are, as in the example of Section 6.3, α (anchor), $\beta_0, \beta_1, \beta_2$ (root), γ, δ (tail).

7.2 Level 0

$$\Phi_2 = \mathrm{SeedOnly}_0 \vee \exists s_0\, \exists a_0.\ \mathrm{Down}_0(\Phi_1')(s_0, a_0).$$

Dead does not contain the empty word, so $\mathrm{SeedOnly}_0$ is false. The only nonempty slices are those with $s_0 = \sigma$. A cap row does not fire with an empty tail, so the subtraction removes nothing.

Slice	Words
$(\sigma, 1)$	$\sigma\,0^n11$, $\sigma\,0^n101$, $\sigma\,0^n111$
$(\sigma, 0)$	$\sigma\,0^n100$, $\sigma\,0^n110$

Below we follow the slice $(\sigma, 1)$.

7.3 Level 1 (slice $(\sigma, 1)$)

Decomposing by $a_0 = 1$, the candidates become words of the following three kinds ($D_1 = \Sigma^*$).

Original word	Decomposition	Word of level 1 (seed | body, length of body)
0^n101	$0^n \cdot 1 \cdot 0 \cdot 1$	$[0^n] \mid [0]$ (length 1)
0^n11	$0^n \cdot 1 \cdot \varepsilon \cdot 1$	$[0^n] \mid [\,]$ (length 1)
0^n111	$0^n \cdot 1 \cdot \varepsilon \cdot 1 \cdot \varepsilon \cdot 1$	$[0^n] \mid [\,]\,[\,]$ (length 2)

$$\Phi_1' = \mathrm{SeedOnly}_1 \vee \exists s_1\, \exists a_1.\ \mathrm{Down}_1(\Phi_0'')(s_1, a_1).$$

No word has an empty body, so $\mathrm{SeedOnly}_1$ is false.

- The slice $(s_1, a_1) = ([0^n], [0])$: the word $[0^n] \mid [0]$ fires, under the new row $(T', H^{(\alpha,\beta_1,\beta_1,\delta,1)})$, at position $(0, 1)$ (the example of Section 6.3). This body of level 1 has length 1, so this is a firing with an empty tail, and the word is removed by the subtraction. The $\mathcal{C}^0$ of this slice becomes empty. This corresponds to the original word 0^n101 being covered.
- The slice $(s_1, a_1) = ([0^n], [\,])$: the words $[0^n] \mid [\,]$ and $[0^n] \mid [\,]\,[\,]$ have no firing with an empty tail under any new row. The former was shown in the example of Section 6.3. For the latter, the possible positions are $(0, 2)$ and $(1, 2)$. Suppose it fires at $(0, 2)$. Then $H([0^n], [\,])$ requires $r_1 = \beta_1$. However, T' must then read the intermediate block $U_2 = [\,]$ by $\mathrm{Run}_{\beta_1,q}([\,])$, and T cannot, from β_1, read $\varepsilon \cdot 1 = 1$. At $(1, 2)$, T' must read the first block $U_1 = [0^n]$ by $\mathrm{Run}_{\alpha,q}([0^n])$, but T cannot, from α, read 0^n1 (α reads only 0 and L). Hence both words remain in $\mathcal{C}^0$. Moreover, the latter does not fire at the nonempty tail position $(0, 1)$ either: since there is no intermediate block, T' requires $t_0 = r_1$, and $H([0^n], [\,])$ forces $r_1 = \beta_1$, but from β_1 it cannot read R. Hence, by the correspondence of firing in Section 6.3, this word does not fire at level 2 either.

7.4 Level 2 (slice $([0^n], [\,])$)

We decompose by $a_1 = [\,]$ (the empty block, one letter of level 1). The alphabet of level 2 is $D_2 = D_1^*$. Each letter occurring in the encoding of this slice is a sequence that does not contain the letter $[\,]$ of level 1 (the condition NoA).

Body of level 1	Decomposition	Word of level 2 (seed | body, length of body)
$[\,]$	$(\,) \cdot [\,]$	$(\,) \mid$ (empty; length 0)
$[\,]\,[\,]$	$(\,) \cdot [\,] \cdot (\,) \cdot [\,]$	$(\,) \mid (\,)$ (length 1)

Here $(\,)$ is a sequence containing no letter of level 1 (one letter of level 2). Φ_0'' is the formula of Theorem 6.8. The first word consists of a seed only, so $\mathrm{SeedOnly}_2$ is true. This corresponds to the original word 0^n11 being an avoider. The second word (the original word 0^n111) also has a body of rank 0 and no row fires on it, so it is also a solution on the side of NonEmpty.

7.5 Conclusion

Put $M_1 = M'[\sigma, 1]$, $M_2 = M_1'[[0^n], [\,]]$. Since $M_2 \models \Phi_0''$, by lowering and existential quantification we have $M_1 \models \Phi_1'$, and further $\Sigma \models \Phi_2$. Hence AvoidExists(E) is true, and E is not universal. If, as a process separate from the decision, we examine words in order of length (Corollary 6.26), the first avoider is 11, and sabBound(E) = 2. In the slice $(\sigma, 0)$, the words 0^n100, 0^n110 are also avoiders, and they can be followed in the same way.

7.6 An example that is universal

To C we add 0^*11 and 0^*100, taking

$$C_1 = 0^*1\Sigma^3, \quad C_2 = 0^*11, \quad C_3 = 0^*100$$

and then Cand $= 0^* \cup 0^*1 \cup 0^*10 \cup 0^*101$, Dead $= 0^*101$ ($n_{\text{acc}} = 4$, $h = 2$). At level 1, the word of the unique nonempty slice, $[0^n] \mid [0]$, is removed by the subtraction, and $\mathcal{C}^0$ becomes empty. At every level, SeedOnly is false, so Φ_2 is false, and E is universal. This conclusion is not obtained by examining finitely many words. It is guaranteed for words of all lengths by Theorem 6.24 and Theorem 6.25.

7.7 The skeleton of the assembled formula

The sentence Φ_2 that the procedure actually produces is large, but extracting only the part that becomes true gives the following. Each line gives the meaning of the formula of the line above, taken one stage down by lowering.

Level	Part that becomes true (schematic)	Meaning
2 (sentence of M_2)	SeedOnly_2	The candidates of level 2 contain a word of length 1 (a word consisting of a seed only)
1 (formula of M_1)	$\exists s_1\, \exists a_1.\ \text{Down}_1(\text{SeedOnly}_2)(s_1, a_1)$	For some letters of level 1, s_1, a_1, the word whose body consists of exactly one a_1, namely $s_1 \mid a_1$, belongs to $\mathcal{C}^0_{s_1,a_1}$ (evidence: $s_1 = [0^n]$, $a_1 = [\,]$)
0 (formula of Σ)	Of $\exists s_0\, \exists a_0.\ \text{Down}_0(\cdots)(s_0, a_0)$, the branch $s_0 = \sigma \wedge a_0 = 1$	For some n, $\sigma\, 0^n 11$ belongs to the $(\sigma, 1)$ slice of level 0, and has no firing with an empty tail at level 0 or at level 1

The formula of the last line, which does not contain names such as Run, H, NoA, is a formula of Σ, and it can be evaluated by expanding the quantifiers over the finitely many letters of $\Sigma = \{0, 1\}$. That formula has the form of a nonemptiness formula. As semantic evidence for this branch we can choose $n = 0$, that is, the word 11, and this is the shortest avoider. However, this does not show the concrete syntax of the generated formula. The actual formula is determined by the simple paths and the edge conditions of the automata assembled at each level (after products, determinization and projection). The table above rereads one of the disjuncts that become true in that evaluation, as a meaning at each level. Other branches can also be true. For example, in the branch $a_0 = 0$, 100 becomes, via $[1] \mid [\,]$, a word consisting of a seed only at level 2, and it satisfies the same SeedOnly_2.

7.8 A small example computed to the end

The example above has two levels, and it is too large for us to write out the whole generated formula. We therefore follow the automata and formulas to the end for a small input whose encoding needs only one stage ($h = 1$). The formulas shown here are the formulas obtained by the construction, simplified by hand to an equivalent form.

Input. Let $\Sigma = \{0, 1\}$, let the only cap component be $A = \{\varepsilon\}$, $B = \{1\}$, and let $C = \{0\} \cup 1\Sigma^2$. The cap term is $\text{pr}(1^*) = 1^*$, and $C\Sigma^*$ is the set of "words starting with 0, or words starting with 1 of length at least 3". Hence

$$\text{Cand} = \{\varepsilon, 1, 10, 11\}, \qquad \text{Mid} = \{\varepsilon, 1\}.$$

The accepting states of the DFA of Cand are the three states "nothing read", "1 read", and "two letters read", so $n_{\text{acc}} = 3$, $h = 1$. No word can be extended infinitely, so Live $= \emptyset$, and the inclusion test passes trivially. We have Dead $= \{10, 11\}$ (with rank 0 and 1 respectively). To give the answer first, 11 is covered by the cut $(0, 1)$ (root 1, tail 1), and 10 is an avoider.

The cap row. For the cap row $(T, \top)$, we define T by the states ι (initial), α, β_0, β_1, γ, δ (accepting) and the edges

$$\iota \xrightarrow{\top} \alpha \text{ (seed)}, \quad \alpha \xrightarrow{L} \beta_0, \quad \beta_0 \xrightarrow{x=1} \beta_1, \quad \beta_1 \xrightarrow{R} \gamma, \quad \gamma \xrightarrow{\top} \delta, \quad \delta \xrightarrow{\top} \delta$$

(the anchor is empty, the root is 1, the tail has at least one letter).

Level 0. The candidates are $\{\sigma 10, \sigma 11\}$. There is no word of length 1, so SeedOnly_0 is false. A cap row has no firing with an empty tail, so $F_{s,a}$ is empty. The nonempty slices are $(\sigma, 0) = \{\sigma 10\}$ and $(\sigma, 1) = \{\sigma 11\}$.

Encoding. In $(\sigma, 1)$ we have $11 = \varepsilon \cdot 1 \cdot \varepsilon \cdot 1$, so the encoding is $[\,] \mid [\,]$ (seed $[\,]$, body of length 1). In $(\sigma, 0)$ we have $10 = 1 \cdot 0$, so the encoding is $[1]$ (seed only, empty body).

Computing the guard H. For 11, the cut $(0, 1)$ corresponds to $U_1 = U_2 = \varepsilon$, $\pi = \pi' = S = \varepsilon$. The index of the corresponding new row is $(r_0, r_1, t_0, t_1, \iota) = (\alpha, \beta_1, \beta_1, \delta, 1)$. If we evaluate $H^{(\alpha,\beta_1,\beta_1,\delta,1)}(U, V)$ at $U = V = \varepsilon$, there are no letters on the tapes, and we are in the case of the proof of Lemma 6.15 labeled "$\pi = \pi' = \varepsilon$". The conditions are that T, from α, reads $L\,a$ and arrives at β_1 ($\alpha \xrightarrow{L} \beta_0 \xrightarrow{a} \beta_1$, that is, $a = 1$), that from β_1 it reads $R\,a$ and arrives at δ ($\beta_1 \xrightarrow{R} \gamma \xrightarrow{a} \delta$, always possible), and the guard $\top$. Hence

$$H^{(\alpha,\beta_1,\beta_1,\delta,1)}(\varepsilon, \varepsilon) \iff a = 1$$

and this is a formula, in the parameter a only, of Σ.

The formula of level 1. Since $h - 1 = 0$, at level 1 we build the formula of Theorem 6.8

$$\Phi'_0 = \text{SeedOnly}_1 \vee \exists s_1\, \exists a_1.\ \text{NonEmpty}_{\mathcal{C}^0_{s_1,a_1}}(s_1, a_1)$$

(a sentence of $M_1 = M'[s_0, a_0]$).

Translation of SeedOnly_1. The candidates of level 1 have a word of length 1 exactly when a body of level 0 enters, in the form $U\,a_0$ (U does not contain a_0), the candidates $\sigma\,\text{Dead}$. The DFA of the body used here recognizes Dead. Its states are q_0 (nothing read), q_1 (1 read), q_2 (two letters read), with $q_0 \xrightarrow{1} q_1 \xrightarrow{0,1} q_2$, and the only accepting state is q_2 (unlike the DFA of Cand, q_0, q_1 are not accepting). Since this DFA accepts exactly the words $1x$ with $x \in \{0, 1\}$, the only body of the form $U\,a_0$ with a_0 not occurring in U is $U = 1$, $a_0 = 0$. We now check that the translation finds this. Translating SeedOnly_1 gives an automaton that reads U in reverse, and the transition reading the final a_0 is absorbed into the initial condition. Reading from the end, the run starts in the state from which a_0 leads to the accepting state. It then follows the transitions of U backwards and must finish in the initial state q_0, where the seed is checked. Simplifying this gives the following automaton recognizing the same language: the initial condition $\text{Init}_{q_1} = (a_0 = 0 \vee a_0 = 1)$ (from q_1 we can read a_0 and arrive at q_2), the edge $q_1 \xrightarrow{x=1 \wedge x \neq a_0} q_0$ (reading the letters of U backwards), and the acceptance condition $\text{Fin}_{q_0} = (s_0 = \sigma)$ (the seed condition; it comes from the condition on the first sequence of the candidates, $\exists y.\,(y = \sigma \wedge y = s_0)$). Projecting out the tape of U and writing nonemptiness as a formula gives

$$\text{Down}_0(\text{SeedOnly}_1)(s_0, a_0) \equiv (s_0 = \sigma) \wedge \exists x.\,(x = 1 \wedge x \neq a_0) \wedge (a_0 = 0 \vee a_0 = 1)$$

(the path of length 0, corresponding to $U = \varepsilon$, does not contribute, since q_1 is not accepting). This is equivalent to $(s_0 = \sigma) \wedge (a_0 = 0)$.

The side of NonEmpty. The only word of level 1 with a nonempty body is, at $(s_0, a_0) = (\sigma, 1)$, the word $[\,] \mid [\,]$. This word, under the new row with index $(\alpha, \beta_1, \beta_1, \delta, 1)$, fires with an empty tail at position $(0, 1)$. Indeed, T' proceeds as before$(1, \alpha) \to$ waitLeft $\xrightarrow{L}$ middle$(\beta_1) \to$ waitRight $\xrightarrow{R}$ after(δ) (the two unmarked arrows each consume an empty block as one letter). Start_α (T reads the seed σ and arrives at α) and Final_δ are true, and no Run reading an intermediate block is needed. The guard is $H(s_1, a_1)$ (since $j = 0$, the left argument is the seed s_1), which for $s_1 = a_1 = [\,]$ is equivalent to $a_0 = 1$, and it holds on this branch. Hence the only word with a nonempty body is always subtracted by the firing of this row, and $\mathcal{C}^0$ becomes empty. The branch $a_0 = 0$ has no word with a nonempty body. Hence $\exists s_1\, \exists a_1.\ \text{NonEmpty}_{\mathcal{C}^0_{s_1,a_1}}$ is false, and by the equivalence of lowering, the formula translating it is also equivalent to false.

The sentence of level 0 and its evaluation. Putting the above together,

$$\Phi_1 \equiv \text{SeedOnly}_0 \vee \exists s_0\, \exists a_0.\ \Big((s_0 = \sigma) \wedge \exists x.\,(x = 1 \wedge x \neq a_0) \wedge (a_0 = 0 \vee a_0 = 1)\Big)$$

and SeedOnly_0 is false. This contains neither Run nor H, is a sentence of Σ, and can be evaluated by substituting the letters of $\Sigma = \{0, 1\}$: it is true with $s_0 = \sigma$, $a_0 = 0$, $x = 1$. Hence E is not universal, and the true branch $a_0 = 0$ corresponds to the avoider 10.

The universal case. To C we add 10, so that Cand $= \{\varepsilon, 1, 11\}$. Then Dead $= \{11\}$. From the DFA of the body, the edge "1 read" $\xrightarrow{0}$ disappears, so the translation of SeedOnly_1 becomes $(s_0 = \sigma) \wedge \exists x.\, (x = 1 \wedge x \neq a_0) \wedge (a_0 = 1)$, which is false since $x = 1 \neq a_0 = 1$ is a contradiction. The side of NonEmpty is false as above. Hence Φ_1 is false, and E is universal.

8 Formalization in Lean

8.1 Formalization

The main theorem of this paper (Theorem 6.27) and the main lemmas leading to it are formalized in Lean 4. The formalization is in `lean/` of the repository and can be checked with `lake build`. All final theorems depend only on the standard Lean axioms `propext`, `Classical.choice`, `Quot.sound`. `sorry`, `admit`, `axiom`, `native_decide`, `implemented_by`, `unsafe` do not appear in any module.

The formalization also re-proves the reduction of Section 4 (the forward direction) independently of [1]. In the process, a procedure computing a countermodel has also been obtained for all three classes of trees. For finite trees, if the entailment does not hold, it returns a valuation in finite trees that satisfies φ and violates $a \leq b$. For regular and arbitrary trees, it returns a finite labeled graph and a root state for each variable (whose unfolding satisfies φ and violates $a \leq b$). In each case one path where the violation occurs is returned, and it corresponds to an avoider of the left or the right restricted cap set expression in the sense of Section 4.

The correspondence between the main claims and the Lean theorems is as follows (module names omit `NSSE.`).

Claim	Lean module / theorem
Internals of the reduction of Section 4 (terms whose proofs are omitted in this paper: safety, side automaton, soundness, completeness; see the corresponding sections of [1])	`FiniteSubtypeSafety`, `ForwardTableAutomaton`, `FiniteForwardSoundness`, `FiniteRightRecognition`, `RegularSubtypeRecognition`
Normalization to restricted cap set expressions	`ForwardCapNormalization.normalize_correct`
Live lemma (Lemma 5.14)	`FiniteCapLive`
Rank lemma (Lemma 5.21)	`NonliveBorderRank.nonliveDFA_rank_bound`
Strict descent (Lemma 6.11)	`TerminalBorderRank.rank_add_one_le_decoded`
Guard H, outside controller, gluing	`GuardedBlockRelation`, `OutsideBlockController`, `BlockEndpointGluing`, `BlockCandidateTransport`
Firing with an empty tail and its subtraction	`EmptyTailAutomaton`, `EmptyTailRemoval`
Uniform rank theorem (Theorem 6.24)	`RankAvoidanceCompiler`, `FiniteRankDecision`
Lowering lemma (Lemma 6.21)	`SymbolicAutomata`, `ConvolutionShapes`, `SymbolicProjection`, `SymbolicPreimage`
Decision of universality (Theorem 6.25)	`FiniteCapLive.decideUncovered_correct`
Short-avoider bound (Corollary 6.26)	`FiniteCapSAB.sabBound_original`
Main theorem (Theorem 6.27), countermodel	`TermEntailmentDecision`, `TermCountermodelExtraction`

For example, the main theorem for finite trees is stated in Lean in the form

$$\texttt{decideFinite}\ Q\ \texttt{hd} = \texttt{true} \iff \varphi \models_{\mathrm{Fin}} a \leq b \qquad (\texttt{decideFinite_correct})$$

Here Q is the input query (the syntactic conditions on terms and constraints are built into its type), and `hd` is a proof of $0 < d$. Apart from the syntactic conditions on the input and the positive arity, there are no additional assumptions such as satisfiability.

9 Conclusion

We showed that, in the case of one constructor (of arity $d \geq 1$, the covariant f), non-structural subtype entailment is decidable for each of finite, regular and arbitrary trees. The proof consists of the satisfiability test for the premise, the reduction of Niehren and Priesnitz [1] (in the satisfiable case) to the universality of restricted cap set expressions, and a decision procedure for that universality. The decision procedure has the following three main points.

- The border condition is the only non-regular element.
- Whether all live words are covered is equivalent to coverage by roots of bounded length, and this reduces to an inclusion test for regular languages.
- In the nonlive part the rank (the maximum number of nontrivial borders of each suffix) is at most $\max(0, n_{\mathrm{acc}} - 2)$, and after peeling it off one level at a time by block encoding, the problem is reduced by lowering to the evaluation of a first-order sentence over a finite alphabet.

The result is formalized in Lean.

This paper treats the case of one covariant constructor. Decidability for a single constructor with arbitrary variance, and for several constructors with arbitrary variance, is outside the scope of this paper. Those results were obtained after the Japanese draft of this paper and are formalized in Lean in the same repository; they have not been externally reviewed.

References

[1] J. Niehren, T. Priesnitz. Non-structural subtype entailment in automata theory. *Information and Computation* 186 (2003) 319–354.

[2] A. Bès. An application of the Feferman–Vaught theorem to automata and logics for words over an infinite alphabet. *Logical Methods in Computer Science* 4(1:8), 2008.

[3] F. Henglein, J. Rehof. Constraint automata and the complexity of recursive subtype entailment. ICALP 1998.

[4] Z. Su, A. Aiken, J. Niehren, T. Priesnitz, R. Treinen. The first-order theory of subtyping constraints. POPL 2002.